

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

HỒ NHẬT MINH

ĐỒ ÁN CHUYÊN NGÀNH

Thiết kế và Triển khai Quy trình
DevSecOps Kết hợp GitOps
cho Hệ thống Microservices trên Nền tảng
AWS

Design and Implementation of a DevSecOps Pipeline
Combined with GitOps for a Microservices System on
AWS

KỸ SƯ NGÀNH MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

TP. HỒ CHÍ MINH, 2026

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

HỒ NHẬT MINH – 23520924

ĐỒ ÁN CHUYÊN NGÀNH

Thiết kế và Triển khai Quy trình
DevSecOps Kết hợp GitOps
cho Hệ thống Microservices trên Nền tảng
AWS

Design and Implementation of a DevSecOps
Pipeline Combined with GitOps
for a Microservices System on AWS

KỸ SƯ NGÀNH MẠNG MÁY TÍNH VÀ TRUYỀN THÔNG

GIẢNG VIÊN HƯỚNG DẪN

LÊ ANH TUẤN

TP. HỒ CHÍ MINH, 2026

THÔNG TIN HỘI ĐỒNG CHẤM ĐỒ ÁN CHUYÊN NGÀNH

Hội đồng:
Chủ tịch:
Thư ký:
Ủy viên:
Ngày bảo vệ:
Điểm:

LỜI CẢM ƠN

Em xin bày tỏ lòng biết ơn sâu sắc đến thầy Lê Anh Tuấn, giảng viên khoa Mạng Máy Tính và Truyền Thông, Trường Đại học Công nghệ Thông tin – Đại học Quốc gia TP. Hồ Chí Minh. Thầy đã tận tình hướng dẫn, định hướng đề tài, góp ý chuyên môn và động viên em trong suốt quá trình thực hiện đồ án. Những buổi gặp trao đổi với thầy đã giúp em hiểu sâu hơn về các khái niệm DevSecOps, GitOps, cũng như cách triển khai chúng một cách thực tiễn trên nền tảng AWS.

Em cũng xin cảm ơn các thầy cô trong Khoa Mạng Máy Tính và Truyền Thông đã trang bị cho em nền tảng kiến thức về mạng máy tính, bảo mật, điện toán đám mây và phát triển phần mềm – những kiến thức thiết yếu để em có thể thực hiện đồ án này.

Xin cảm ơn gia đình và bạn bè đã luôn ủng hộ, động viên và tạo điều kiện tốt nhất để em hoàn thành đồ án.

TP. Hồ Chí Minh, ngày ... tháng ... năm 2026

Sinh viên thực hiện

Hồ Nhật Minh

MỤC LỤC

Lời cảm ơn	4
Mục lục	5
Danh mục hình	7
Danh mục bảng	8
Danh mục từ viết tắt	9
Tóm tắt đồ án	1
1 Mở đầu	2
1.1 Lý do chọn đề tài	2
1.2 Mục tiêu đồ án	2
1.3 Đối tượng và phạm vi đồ án	3
1.4 Phương pháp thực hiện	3
2 Tổng quan	4
2.1 Kiến trúc Microservices	4
2.2 DevSecOps	5
2.3 GitOps	7
2.4 Nền tảng Kubernetes và Amazon EKS	10
2.5 Infrastructure as Code và Terraform	10
2.6 Các công cụ khác	11
2.7 Policy as Code với Kyverno	13
2.8 Policy as Code với Kyverno	13
2.9 Sample Application: Retail Store	13
3 Phương pháp và Thiết kế Hệ thống	15
3.1 Kiến trúc tổng thể	15
3.2 Cấu trúc Workspace đồ án	17
3.3 Thiết kế Hạ tầng AWS	17
3.4 Thiết kế Security Pipeline	20
3.5 Thiết kế GitOps Pipeline	22
3.6 Thiết kế ECR Repositories	23
4 Kết quả Triển khai và Kiểm thử	25
4.1 Triển khai hạ tầng	25

4.2	Kiểm tra ECR Repositories	27
4.3	Kiểm tra trên AWS Console	28
4.4	Triển khai CI Pipeline	28
4.5	Triển khai CD GitOps với Argo CD	32
4.6	Triển khai Observability	34
4.7	Triển khai ứng dụng qua GitOps Pipeline	35
4.8	Triển khai Policy as Code với Kyverno	36
4.9	Kiểm tra bảo mật Infrastructure as Code với Terrascan	37
4.10	Đánh giá kết quả đạt được	39
5	Kết luận và Hướng phát triển	40
5.1	Kết quả đã đạt được	40
5.2	Những đóng góp của đề án	41
5.3	Hạn chế	41
5.4	Hướng phát triển	41
	Tài liệu tham khảo	43
	Phụ lục	45

DANH MỤC HÌNH

2.1	Sự tiến hóa từ DevOps sang DevSecOps	5
2.2	Mô hình hoạt động của GitOps	8
3.3	Kiến trúc tổng thể DevSecOps + GitOps	16
3.4	Thiết kế CI pipeline với 3 security gates	21
3.5	Mô hình 2-repository GitOps	22

DANH MỤC BẢNG

2.2	Bốn loại kiểm thử bảo mật trong DevSecOps	6
2.3	Các microservice trong Retail Store Sample App	13
3.4	Cấu trúc workspace đồ án	17
3.5	Thiết kế subnet trong VPC	17
3.6	Cấu hình VPC đã triển khai	18
3.7	Cấu hình EKS cluster	19
3.8	Danh sách Kubernetes addons đã cài đặt	19
3.9	Ngưỡng cho từng security gate	22
4.10	Kết quả Terraform Apply	25
4.11	ECR repositories đã tạo	28
4.12	Các stage trong Jenkins CI Pipeline	30
4.13	Kustomize base & overlay manifests	32
4.14	Cấu hình Argo CD Application cho môi trường dev	32
4.15	Các thành phần monitoring và platform đã triển khai	34
4.16	Trạng thái triển khai 5 microservice sau CI/CD hoàn tất	36
4.17	Kyverno ClusterPolicies đã triển khai	37
4.18	Kết quả Terrascan scan	37
4.19	Đánh giá kết quả theo tiêu chí	39
5.20	Danh sách evidence cần thu thập	48

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Nghĩa tiếng Anh / Giải thích
ADOT	AWS Distro for OpenTelemetry
ALB	Application Load Balancer
ALB	Application Load Balancer
AMI	Amazon Machine Image
ANSI	American National Standards Institute
AWS	Amazon Web Services
CD	Continuous Delivery / Continuous Deployment
CI	Continuous Integration
CNI	Container Network Interface
CVE	Common Vulnerabilities and Exposures
DAST	Dynamic Application Security Testing
DDB	Amazon DynamoDB
DNS	Domain Name System
EC2	Amazon Elastic Compute Cloud
ECR	Amazon Elastic Container Registry
EKS	Amazon Elastic Kubernetes Service
ENI	Elastic Network Interface
GitOps	Operations by Git (Git-based operations model)
IAM	Identity and Access Management
IaC	Infrastructure as Code
IEEE	Institute of Electrical and Electronics Engineers
IRSA	IAM Roles for Service Accounts
KMS	Key Management Service
KLTN	Khóa luận tốt nghiệp
ML	Machine Learning
MLOps	Machine Learning Operations
NAT	Network Address Translation
NLB	Network Load Balancer
NLP	Natural Language Processing
OIDC	OpenID Connect
OTEL	OpenTelemetry

OWASP	Open Web Application Security Project
RBAC	Role-Based Access Control
S3	Amazon Simple Storage Service
SAST	Static Application Security Testing
SCA	Software Composition Analysis
SG	Security Group
SNS	Amazon Simple Notification Service
SQS	Amazon Simple Queue Service
TLS	Transport Layer Security
VPC-CNI	Amazon VPC Container Network Interface
VPC	Virtual Private Cloud

TÓM TẮT ĐỒ ÁN

Trong bối cảnh chuyển đổi số, các tổ chức đang dịch chuyển từ kiến trúc monolithic sang kiến trúc microservices để đạt được khả năng mở rộng và tốc độ phát triển cao hơn. Việc triển khai và vận hành hàng chục microservice đặt ra thách thức lớn về bảo mật và quản lý cấu hình. Các quy trình triển khai thủ công truyền thống tiềm ẩn nguy cơ misconfiguration, lỗi hỏng bảo mật không được phát hiện kịp thời, và khó khăn trong rollback. Đồ án này hướng tới giải quyết các vấn đề trên thông qua việc thiết kế và triển khai một quy trình DevSecOps kết hợp GitOps cho hệ thống microservices trên nền tảng AWS.

Quy trình được xây dựng bao gồm: (1) Quản lý hạ tầng bằng Infrastructure as Code với Terraform (module hóa VPC, EKS, ECR với 3 môi trường dev/staging/prod); (2) Tích hợp bảo mật “shift-left” vào CI pipeline qua Jenkins với ba security gate là SonarQube (SAST), OWASP Dependency Check (SCA), và Trivy (Container Image Scan); (3) Triển khai tự động theo mô hình GitOps sử dụng Argo CD kết hợp Kustomize và Helm; (4) Chính sách bảo mật Kubernetes với Kyverno (Policy as Code) và kiểm tra bảo mật hạ tầng với Terrascan; (5) Giám sát tập trung với CloudWatch, Prometheus, Grafana và 10 alerting rules.

Toàn bộ hạ tầng được provision tự động trên AWS bao gồm VPC, EKS cluster v1.33, 3 managed node groups (2 t3.medium + 1 t3.large) và 5 ECR repositories. Ứng dụng Retail Store gồm 5 microservice (UI, Cart, Orders, Catalog, Checkout) được triển khai thành công qua pipeline CI/CD tự động sử dụng Jenkins (CI) và Argo CD (CD GitOps). Kết quả triển khai cho thấy toàn bộ 41 pods trên cluster ở trạng thái Running, bao gồm 5 microservice ứng dụng, MySQL catalog-db, các add-on hệ thống (Argo CD, Jenkins, Prometheus, Grafana, Cert Manager, Kyverno, SonarQube), và Load Balancer Controller đang hoạt động ổn định. CI pipeline đã thực thi thành công các Kaniko build, vượt qua 3 security gate, và tự động cập nhật GitOps repo để kích hoạt Argo CD sync.

Đồ án chứng minh tính khả thi của mô hình DevSecOps kết hợp GitOps trong việc tự động hóa quy trình phát triển, kiểm thử bảo mật, Policy as Code và triển khai microservices, đồng thời cung cấp một tài liệu tham khảo thực tiễn cho các tổ chức muốn áp dụng phương pháp này trên AWS.

Từ khóa: DevSecOps, GitOps, Microservices, Kubernetes, AWS, Terraform, Jenkins, Argo CD, CI/CD, Infrastructure as Code, Kyverno, Policy as Code, Prometheus, Grafana, Terrascan.

Chương 1. Mở đầu

1.1. Lý do chọn đề tài

Sự phát triển nhanh chóng của công nghệ điện toán đám mây (Cloud Computing) và kiến trúc microservices đã thay đổi căn bản cách các tổ chức xây dựng và vận hành phần mềm. Thay vì triển khai một ứng dụng monolithic trên máy chủ vật lý, các doanh nghiệp hiện đại phân tách ứng dụng thành hàng chục hoặc thậm chí hàng trăm microservice độc lập, mỗi service đảm nhiệm một chức năng riêng biệt và có thể được phát triển, kiểm thử, triển khai một cách độc lập.

Kiến trúc microservices mang lại nhiều lợi ích: khả năng mở rộng linh hoạt, tốc độ phát triển nhanh hơn, và khả năng chịu lỗi tốt hơn. Tuy nhiên, nó cũng đặt ra những thách thức mới về vận hành (operations). Khi số lượng service tăng lên, việc triển khai thủ công trở nên bất khả thi. Đồng thời, bề mặt tấn công (attack surface) mở rộng đáng kể khi mỗi service có thể là một điểm yếu tiềm tàng.

Hai phương pháp luận hiện đại đã xuất hiện để giải quyết các thách thức này:

DevSecOps là sự kết hợp của Development (Phát triển), Security (Bảo mật), và Operations (Vận hành). Thay vì coi bảo mật là bước kiểm tra cuối cùng trước khi phát hành, DevSecOps tích hợp bảo mật vào mọi giai đoạn của vòng đời phát triển phần mềm – từ lập kế hoạch, viết code, build, kiểm thử, đến triển khai và giám sát. Cách tiếp cận này được gọi là “shift-left security” – đẩy việc kiểm tra bảo mật sang trái trên dòng thời gian phát triển.

GitOps là mô hình vận hành trong đó toàn bộ trạng thái mong muốn của hệ thống được khai báo dưới dạng code trong Git repository. Một operator như Argo CD liên tục so sánh trạng thái thực tế của Kubernetes cluster với trạng thái khai báo trong Git, và tự động đồng bộ khi phát hiện sự khác biệt. Git trở thành “single source of truth” – nguồn sự thật duy nhất của hệ thống.

Mặc dù cả hai phương pháp luận này đều đã được chứng minh hiệu quả trong các tổ chức lớn như Google, Netflix, và Amazon, việc kết hợp chúng thành một quy trình thống nhất, hoàn chỉnh cho microservices trên AWS vẫn là một bài toán chưa có nhiều hướng dẫn thực tiễn, đặc biệt trong môi trường học thuật tại Việt Nam.

Đề án này được thực hiện với mong muốn xây dựng một hệ thống minh họa toàn bộ quy trình DevSecOps kết hợp GitOps, từ khâu quản lý hạ tầng (Infrastructure as Code), đến CI/CD tự động với security gates, và triển khai GitOps trên Kubernetes. Hệ thống sẽ sử dụng toàn bộ công cụ open-source và dịch vụ AWS phổ biến, đảm bảo tính khả thi và khả năng áp dụng thực tế.

1.2. Mục tiêu đề án

Mục tiêu tổng quát của đề án là thiết kế và triển khai thành công một quy trình DevSecOps kết hợp GitOps, vận hành trên nền tảng AWS với một ứng dụng microservices thực tế. Các mục tiêu cụ thể bao gồm:

1. **Tự động hóa hạ tầng bằng Infrastructure as Code:** Toàn bộ hạ tầng AWS (VPC, EKS cluster, IAM roles, ECR repositories) được định nghĩa và quản lý bằng Terraform. State được lưu tập trung trên S3 với cơ chế lock qua DynamoDB.

2. **Tích hợp bảo mật vào CI pipeline (Shift-Left Security):** Xây dựng Jenkins pipeline tự động thực hiện SAST (SonarQube), SCA (OWASP Dependency Check), và Container Image Scan (Trivy) trước khi cho phép deploy. Chỉ những artifact vượt qua tất cả security gate mới được đẩy lên ECR và triển khai.
3. **Triển khai GitOps với Argo CD:** Sử dụng mô hình 2-repo (App Repo + GitOps Repo). Argo CD tự động đồng bộ trạng thái từ GitOps Repo lên EKS cluster. Rollback được thực hiện bằng git revert.
4. **Giám sát tập trung:** Thiết lập CloudWatch Logs cho EKS control plane, Prometheus + Grafana dashboards với 10 alerting rules cho cluster và ứng dụng.
5. **Chứng minh tính khả thi:** Triển khai thành công toàn bộ 5 microservices của ứng dụng bán lẻ mẫu lên EKS thông qua CI/CD pipeline tự động end-to-end (Jenkins → ECR → Argo CD → EKS), thực hiện demo rollback bằng GitOps, và triển khai Policy as Code (Kyverno) cùng IaC security scanning (Terrascan).

1.3. Đối tượng và phạm vi đồ án

Đối tượng nghiên cứu: Quy trình DevSecOps, mô hình GitOps, nền tảng Kubernetes (Amazon EKS), các công cụ CI/CD (Jenkins, Argo CD), công cụ bảo mật (SonarQube, OWASP DC, Trivy, Kyverno, Terrascan), và hạ tầng AWS (VPC, ECR, IAM, CloudWatch).

Phạm vi:

- Hệ thống được triển khai trên một EKS cluster với 3 managed node groups (2 t3.medium + 1 t3.large), không triển khai multi-cluster hay multi-region.
- Chỉ triển khai môi trường dev. Cấu trúc cho staging và prod được chuẩn bị (Kustomize overlays) nhưng không triển khai đầy đủ.
- CI pipeline sử dụng Jenkins với Shared Libraries; CD sử dụng Argo CD kết hợp Kustomize và Helm umbrella chart; không triển khai các công cụ thay thế như GitHub Actions, GitLab CI, hay Flux CD.
- Security scanning bao gồm SAST (SonarQube), SCA (OWASP DC), container scan (Trivy), và IaC scanning (Terrascan); Policy as Code với Kyverno (5 ClusterPolicies); không bao gồm DAST, penetration testing.
- Không triển khai Service Mesh (Istio/Linkerd).

1.4. Phương pháp thực hiện

Đồ án được thực hiện theo phương pháp thực nghiệm (empirical approach), bao gồm các bước:

1. **Nghiên cứu lý thuyết:** Tìm hiểu cơ sở lý luận về DevSecOps, GitOps, Kubernetes, AWS EKS thông qua tài liệu học thuật và tài liệu kỹ thuật chính thức từ AWS.
2. **Khảo sát công cụ:** Đánh giá và lựa chọn các công cụ phù hợp cho từng giai đoạn của pipeline.
3. **Thiết kế kiến trúc:** Vẽ sơ đồ tổng thể hệ thống, thiết kế luồng dữ liệu và cấu trúc workspace.
4. **Triển khai hạ tầng:** Sử dụng Terraform để tự động hóa việc provision tài nguyên AWS.
5. **Xây dựng pipeline:** Cấu hình Jenkins cho CI, Argo CD cho CD GitOps.
6. **Kiểm thử và đánh giá:** Thực hiện kiểm thử end-to-end, đánh giá hiệu năng và tính bảo mật của pipeline.

Chương 2. Tổng quan

Chương này trình bày cơ sở lý thuyết và các công nghệ nền tảng liên quan đến đề án, bao gồm kiến trúc microservices, phương pháp DevSecOps, mô hình GitOps, nền tảng Kubernetes và các dịch vụ AWS được sử dụng.

2.1. Kiến trúc Microservices

2.1.1. Từ Monolithic đến Microservices

Trong mô hình phát triển phần mềm truyền thống, ứng dụng được xây dựng dưới dạng một khối thống nhất (monolithic) – tất cả các chức năng được đóng gói trong cùng một codebase, build thành một artifact duy nhất và triển khai trên một hoặc một vài máy chủ. Mô hình này có ưu điểm là đơn giản trong phát triển ban đầu và dễ kiểm thử tích hợp. Tuy nhiên, khi ứng dụng phát triển về quy mô, các hạn chế trở nên rõ ràng:

- **Scale không linh hoạt:** Khi cần mở rộng, toàn bộ ứng dụng phải được nhân bản, ngay cả khi chỉ một thành phần bị quá tải.
- **Phát triển chậm:** Nhiều team cùng làm việc trên một codebase dẫn đến xung đột thường xuyên và thời gian build kéo dài.
- **Công nghệ bị khóa cứng:** Toàn bộ ứng dụng phải dùng chung một ngôn ngữ lập trình và framework.
- **Triển khai rủi ro cao:** Một thay đổi nhỏ có thể yêu cầu triển khai lại toàn bộ ứng dụng.

Kiến trúc microservices giải quyết các hạn chế này bằng cách phân tách ứng dụng thành nhiều service nhỏ, độc lập. Mỗi service:

- Đảm nhiệm một chức năng nghiệp vụ cụ thể (single responsibility).
- Có thể được phát triển bằng ngôn ngữ lập trình và framework riêng (polyglot).
- Có database riêng (database per service pattern).
- Có thể được triển khai, scale, và cập nhật độc lập.
- Giao tiếp với nhau qua API (REST, gRPC) hoặc message queue.

2.1.2. Thách thức của Microservices

Bên cạnh những lợi ích, kiến trúc microservices đặt ra những thách thức không tồn tại trong mô hình monolithic:

- **Độ phức tạp trong vận hành:** Quản lý hàng chục service yêu cầu công cụ orchestration mạnh mẽ, thường là Kubernetes.
- **Bảo mật phân tán:** Mỗi service là một điểm entry tiềm năng; số lượng điểm cần bảo vệ tăng theo số lượng service.
- **Triển khai phức tạp:** Cần pipeline CI/CD phức tạp hơn để quản lý việc build, test, và deploy nhiều service.
- **Giao tiếp liên service:** Cần cơ chế service discovery, load balancing, circuit breaking, và retry logic.

- **Tính nhất quán dữ liệu:** Mỗi service có database riêng dẫn đến thách thức về distributed transactions.
- **Giám sát và logging:** Cần hệ thống giám sát tập trung để theo dõi trạng thái của tất cả service.

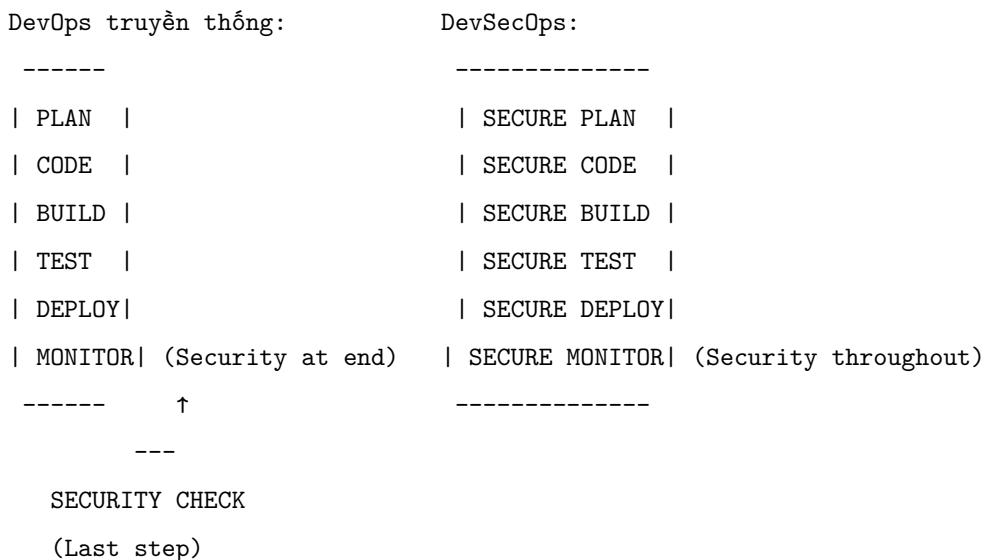
Chính những thách thức này là động lực cho sự phát triển của các phương pháp luận như DevSecOps và GitOps.

2.2. DevSecOps

2.2.1. Khái niệm

DevOps là một tập hợp các thực hành kết hợp giữa Development (phát triển) và Operations (vận hành) nhằm rút ngắn vòng đời phát triển hệ thống và cung cấp tính năng, sửa lỗi, cập nhật thường xuyên. Các nguyên tắc cốt lõi của DevOps bao gồm tự động hóa, đo lường, chia sẻ kiến thức và văn hóa hợp tác giữa các team.

Hình 2.1 minh họa sự tiến hóa từ DevOps truyền thống sang DevSecOps, trong đó bảo mật được tích hợp vào mọi giai đoạn.



Hình 2.1: Sự tiến hóa từ DevOps sang DevSecOps

DevSecOps mở rộng DevOps bằng cách tích hợp bảo mật (security) vào mọi giai đoạn của quy trình phát triển. Triết lý cốt lõi là “shift-left security” – dời việc kiểm tra bảo mật sang trái trên timeline, càng sớm càng tốt. Điều này có nghĩa là:

- Thay vì kiểm tra bảo mật ở cuối pipeline (ngay trước khi deploy), bảo mật được kiểm tra liên tục trong suốt quá trình phát triển.
- Developer được trao quyền và trách nhiệm về bảo mật code của mình.
- Các security check được tự động hóa và tích hợp vào pipeline CI/CD.

2.2.2. Các loại Security Testing trong DevSecOps

Có bốn loại kiểm thử bảo mật chính trong quy trình DevSecOps:

Phương pháp	Tên đầy đủ	Mô tả
SAST	Static Application Security Testing	Phân tích mã nguồn tĩnh – quét source code để tìm lỗ hổng mà không cần chạy ứng dụng. Công cụ phổ biến: SonarQube, Checkmarx, Fortify.
SCA	Software Composition Analysis	Phân tích thành phần phần mềm – kiểm tra lỗ hổng trong các thư viện và dependency bên thứ ba. Công cụ: OWASP Dependency Check, Snyk.
Container Scan	Container Image Scanning	Quét Docker image để tìm CVE trong base image và các package cài đặt. Công cụ: Trivy, Clair.
DAST	Dynamic Application Security Testing	Phân tích ứng dụng đang chạy – tấn công mô phỏng để tìm lỗ hổng thực tế. Không được thực hiện trong phạm vi đề án này.

Bảng 2.2: Bốn loại kiểm thử bảo mật trong DevSecOps

2.2.3. Các công cụ Security sử dụng trong đề án

SonarQube là một nền tảng mã nguồn mở để kiểm tra chất lượng mã nguồn liên tục. Nó thực hiện SAST bằng cách phân tích mã nguồn với hàng trăm quy tắc (rules) để phát hiện bugs, code smells, và lỗ hổng bảo mật. SonarQube hỗ trợ hơn 25 ngôn ngữ lập trình bao gồm Java, TypeScript, Go – phù hợp với sample app của đề án. Kết quả phân tích được hiển thị trên dashboard với các chỉ số như Quality Gate status, Security Rating, Reliability Rating.

OWASP Dependency Check là công cụ SCA do OWASP phát triển. Nó quét các file cấu hình dependency (pom.xml cho Java Maven, package.json cho Node.js, go.mod cho Go) và đối chiếu với cơ sở dữ liệu National Vulnerability Database (NVD) để xác định các dependency có CVE đã biết. Công cụ tạo báo cáo hiển thị CVSS score và mức độ nghiêm trọng của từng lỗ hổng.

Trivy là công cụ quét bảo mật toàn diện do Aqua Security phát triển, được thiết kế để quét Docker images, file system, và Git repository. Trivy có thể phát hiện lỗ hổng trong OS packages (Alpine, Debian, Amazon Linux) và language-specific packages (Java JAR, Node.js npm, Go modules). Điểm mạnh của Trivy là tốc độ quét nhanh và tích hợp dễ dàng vào CI pipeline.

Kyverno là công cụ Policy as Code cho Kubernetes, được phát triển bởi Nirmata và hiện là dự án CNCF Incubating. Không giống như OPA/Gatekeeper sử dụng ngôn ngữ Rego, Kyverno cho phép định nghĩa chính sách dưới dạng Kubernetes-native YAML – cùng ngôn ngữ mà developer đã quen thuộc. Kyverno hoạt động như

một dynamic admission controller trong Kubernetes cluster, intercept các API request (create, update, delete) và kiểm tra chúng với các chính sách đã định nghĩa. Các chính sách có thể thực hiện validate (chặn resource không hợp lệ), mutate (tự động sửa đổi resource), và generate (tự động tạo resource bổ sung). Trong đề án, 5 ClusterPolicy được triển khai bao gồm: yêu cầu pod labels chuẩn, cấm sử dụng tag latest, yêu cầu health probes, yêu cầu resource limits, và cấm chạy container với quyền root.

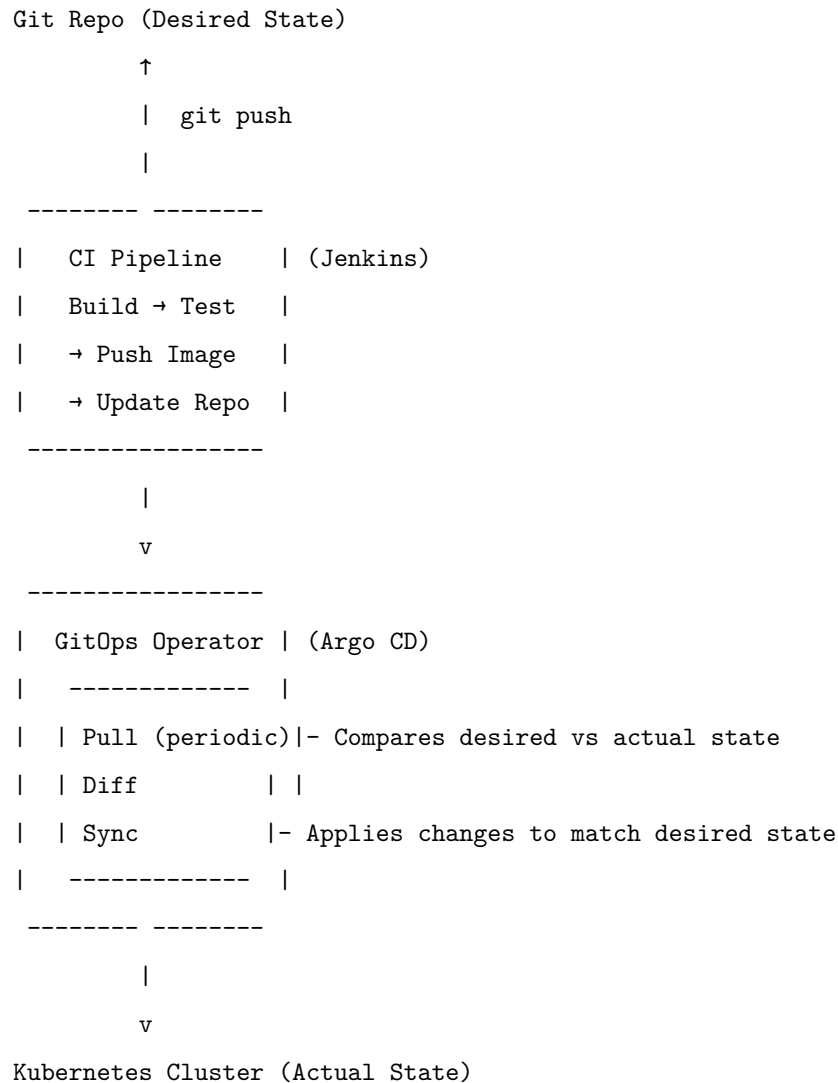
Terrascan là công cụ static code analysis cho Infrastructure as Code, do Tenable phát triển. Terrascan quét các file cấu hình IaC (Terraform, Kubernetes, Helm, Kustomize, Dockerfile) để phát hiện các vi phạm chính sách bảo mật và compliance trước khi provision. Công cụ hỗ trợ hơn 500 chính sách dựa trên các tiêu chuẩn như CIS Benchmarks, NIST, PCI-DSS. Trong đề án, Terrascan được sử dụng để quét toàn bộ Terraform modules trước khi triển khai, phát hiện 1 vi phạm mức LOW (VPC chưa bật flow logs). Song song đó, Checkov (do Bridgecrew/Palo Alto phát triển) cũng được thiết lập sẵn sàng để bổ sung thêm một lớp kiểm tra IaC security.

2.3. GitOps

2.3.1. Khái niệm

GitOps là một mô hình vận hành cho Kubernetes và các nền tảng cloud-native, được Weaveworks đề xuất lần đầu vào năm 2017. Nguyên lý cốt lõi của GitOps là sử dụng Git repository làm “single source of truth” (nguồn sự thật duy nhất) cho trạng thái mong muốn của toàn bộ hệ thống.

Hình 2.2 mô tả luồng hoạt động của GitOps.



Hình 2.2: Mô hình hoạt động của GitOps

GitOps có ba nguyên tắc chính:

1. **Khai báo (Declarative):** Toàn bộ hệ thống phải được mô tả dưới dạng khai báo (declarative configuration), ví dụ như Kubernetes manifests.
2. **Phiên bản hóa (Versioned):** Trạng thái mong muốn được lưu trong Git, tận dụng các tính năng của Git như versioning, branching, pull request review.
3. **Tự động đồng bộ (Automated Sync):** Một operator (software agent) chạy trong cluster liên tục so sánh trạng thái thực tế với trạng thái khai báo trong Git, và tự động đồng bộ (sync) hoặc cảnh báo khi phát hiện sự khác biệt (drift).

2.3.2. Lợi ích của GitOps

- **Audit trail hoàn chỉnh:** Mọi thay đổi đối với hệ thống đều có commit tương ứng trong Git. Có thể dễ dàng truy vết ai đã thay đổi gì, khi nào.
- **Rollback đơn giản:** Rollback = git revert. Không cần quy trình phức tạp, không cần quyền đặc biệt.
- **Bảo mật:** Chỉ cần cấp quyền truy cập Kubernetes cho Argo CD, developer không cần quyền kubectl trực tiếp lên cluster.
- **Tính nhất quán:** Toàn bộ cluster được đồng bộ về đúng trạng thái khai báo trong Git, loại bỏ configuration drift.
- **Tự phục hồi:** Nếu có ai đó thay đổi thủ công (kubectl edit), Argo CD sẽ tự động hoàn nguyên về trạng thái trong Git.

2.3.3. Argo CD

Argo CD là một công cụ GitOps khai báo (declarative GitOps tool) cho Kubernetes, hiện là dự án CNCF Incubating. Argo CD hoạt động theo mô hình pull-based:

1. Argo CD được cài đặt trong Kubernetes cluster.
2. Người dùng định nghĩa Application resource – một Custom Resource Definition (CRD) của Argo CD – chỉ định Git repo, path đến manifests, và cluster đích.
3. Argo CD định kỳ (mặc định mỗi 3 phút) hoặc theo webhook pull Git repo, so sánh với trạng thái thực tế của cluster.
4. Nếu phát hiện sự khác biệt, Argo CD hiển thị trạng thái OutOfSync trên dashboard.
5. Có thể cấu hình auto-sync để Argo CD tự động áp dụng thay đổi.

Argo CD hỗ trợ nhiều định dạng manifest: plain YAML, Kustomize, Helm, Ksonnet, và Jsonnet. Đồ án này sử dụng Kustomize vì tính linh hoạt và là công cụ được tích hợp sẵn trong kubectl.

2.3.4. Hai mô hình triển khai GitOps

Có hai chiến lược triển khai GitOps phổ biến:

1. **Single Repo:** Chỉ có một repository chứa cả mã nguồn ứng dụng và manifest triển khai. Phù hợp với dự án nhỏ, ít service.
2. **Dual Repo** (được sử dụng trong đồ án): Hai repository riêng biệt:
 - **App Repo:** Chứa mã nguồn microservices, Dockerfile, Jenkinsfile. CI pipeline được kích hoạt khi có push vào repo này.
 - **GitOps Repo:** Chứa các manifest triển khai (Kustomize/Helm). CI pipeline tự động cập nhật image tag vào repo này sau khi build thành công. Argo CD theo dõi repo này và đồng bộ lên cluster.

Mô hình dual repo mang lại lợi ích phân tách quyền: developer chỉ cần quyền trên App Repo, không cần truy cập GitOps Repo (vốn do CI pipeline tự động cập nhật). Điều này tăng cường bảo mật và giảm rủi ro.

2.4. Nền tảng Kubernetes và Amazon EKS

2.4.1. Kubernetes

Kubernetes (thường viết tắt là K8s) là một nền tảng mã nguồn mở để tự động hóa việc triển khai, scale, và quản lý các ứng dụng containerized. Kubernetes được Google thiết kế dựa trên kinh nghiệm vận hành hệ thống Borg và được trao cho Cloud Native Computing Foundation (CNCF) quản lý từ năm 2015.

Các thành phần chính của Kubernetes bao gồm:

- **Control Plane:** Bao gồm kube-apiserver, etcd (database phân tán), kube-controller-manager, và kube-scheduler. Quản lý trạng thái toàn cluster và ra quyết định (ví dụ: schedule pod lên node nào).
- **Worker Nodes:** Máy chủ chạy workload thực tế. Mỗi node chạy kubelet (agent giao tiếp với control plane), container runtime (Docker/containerd), và kube-proxy (quản lý network rules).
- **Pod:** Đơn vị triển khai nhỏ nhất trong Kubernetes, có thể chứa một hoặc nhiều container chia sẻ cùng network namespace và storage.
- **Service:** Một trừu tượng định nghĩa tập hợp các pod và policy để truy cập chúng, cung cấp IP ổn định và load balancing.
- **Deployment:** Quản lý việc triển khai và cập nhật pod, hỗ trợ rolling update và rollback.
- **Namespace:** Cơ chế phân vùng logic trong cluster, cho phép chia sẻ một cluster vật lý giữa nhiều team hoặc môi trường.

2.4.2. Amazon EKS

Amazon Elastic Kubernetes Service (EKS) là dịch vụ Kubernetes được quản lý bởi AWS, cung cấp control plane Kubernetes có tính sẵn sàng cao (highly available) trên nhiều Availability Zone. Lợi ích của EKS so với tự quản lý Kubernetes:

- **Control plane được quản lý:** AWS đảm nhận việc vận hành, scale, và cập nhật control plane. Người dùng chỉ cần quản lý worker nodes.
- **Tích hợp sâu với hệ sinh thái AWS:** IAM Roles for Service Accounts (IRSA) cho phép gán IAM role trực tiếp cho Kubernetes service account; AWS Load Balancer Controller tự động tạo ALB/NLB; VPC-CNI cung cấp IP riêng cho mỗi pod.
- **Bảo mật:** Control plane chạy trong VPC riêng của AWS, tích hợp KMS để mã hóa secrets, hỗ trợ private endpoint.
- **Managed Node Groups:** AWS tự động quản lý việc provisioning, scaling, và cập nhật AMI cho worker nodes.

2.5. Infrastructure as Code và Terraform

Infrastructure as Code (IaC) là phương pháp quản lý hạ tầng CNTT thông qua các file cấu hình (code) thay vì thao tác thủ công qua giao diện (Console/CLI). Các nguyên tắc của IaC: idempotency (có thể chạy lại

nhiều lần mà không thay đổi kết quả), declarative (khai báo trạng thái mong muốn, công cụ tự tính toán cách đạt được), và version control (lưu trong Git).

Terraform, phát triển bởi HashiCorp, là công cụ IaC phổ biến nhất hiện nay. Terraform sử dụng ngôn ngữ HCL (HashiCorp Configuration Language) để định nghĩa tài nguyên, state file để theo dõi trạng thái thực tế, và provider để tương tác với các nền tảng khác nhau (AWS, Azure, GCP, Kubernetes, Helm).

Trong đồ án này, Terraform được sử dụng để quản lý toàn bộ hạ tầng AWS và Kubernetes addons. Mã Terraform được tổ chức thành các module độc lập (vpc/, eks/, ecr/) trong thư mục 03-infra/terraform/modules/, mỗi module có biến đầu vào (variables.tf), đầu ra (outputs.tf), và khai báo provider (versions.tf) riêng biệt – tuân theo các best practice về IaC của Terraform Registry:

- **Terraform Backend:** State được lưu trên S3 (có versioning), lock qua DynamoDB, cấu hình qua file backend.hcl.
- **VPC Module:** Sử dụng terraform-aws-modules/vpc/aws để tạo VPC, 6 subnets (3 public + 3 private), route tables, NAT Gateway.
- **EKS Module:** Sử dụng terraform-aws-modules/eks/aws để tạo EKS cluster, 3 managed node groups (2 t3.medium + 1 t3.large), VPC-CNI addon, IRSA, và các Kubernetes addons qua Helm provider.
- **ECR Module:** Module tự xây dựng (modules/ecr/) để tạo 5 ECR repositories cho từng microservice, cấu hình image scanning on push và lifecycle policy.
- Cấu trúc đa môi trường: envs/dev/, envs/staging/, envs/prod/ – mỗi môi trường có terraform.tfvars riêng.

2.6. Các công cụ khác

2.6.1. Jenkins

Jenkins là một automation server mã nguồn mở, được viết bằng Java, là nền tảng CI/CD phổ biến nhất trong giới doanh nghiệp. Jenkins hỗ trợ kiến trúc plugin với hơn 1.800 plugin, cho phép tích hợp với hầu hết các công cụ trong hệ sinh thái DevOps. Trong đồ án này, Jenkins đảm nhiệm vai trò CI server:

- Lắng nghe webhook từ GitHub khi có push vào App Repo.
- Thực thi pipeline được định nghĩa trong Jenkinsfile (Pipeline as Code).
- Pipeline gồm các stage: Checkout → SonarQube Analysis → OWASP Dependency Check → Docker Build → Trivy Image Scan → Push ECR → Update GitOps Repo.
- Sử dụng Shared Libraries để đóng gói các hàm tái sử dụng: awsSetup, buildService, sonarQubeAnalysis, owaspDependencyCheck, dockerBuildAndPush, updateGitOpsRepo – giúp Jenkinsfile chính chỉ còn 120 dòng so với hàng trăm dòng nếu viết trực tiếp.
- Docker images được build bằng Kaniko trong Kubernetes pods (không cần Docker daemon) để tăng tính bảo mật.

2.6.2. Kustomize

Kustomize là một công cụ quản lý cấu hình Kubernetes, được tích hợp sẵn trong kubectl từ phiên bản 1.14. Không giống như Helm (dùng template), Kustomize sử dụng phương pháp patch và overlay – bắt đầu từ một bộ manifest cơ sở (base) và áp dụng các lớp tùy chỉnh (overlay) cho từng môi trường.

Cấu trúc Kustomize trong đồ án:

```
gitops-repo/apps/retail-store/  
  -- base/  
|   -- kustomization.yaml  
|   -- ui-deployment.yaml  
|   -- cart-deployment.yaml  
|   -- orders-deployment.yaml  
|   -- catalog-deployment.yaml  
|   -- checkout-deployment.yaml  
-- overlays/  
  -- dev/  
|   -- kustomization.yaml  
|   -- patches/  
  -- staging/  
  -- prod/
```

2.6.3. Helm

Bên cạnh Kustomize, đồ án còn sử dụng Helm dưới dạng umbrella chart để cung cấp một cách tiếp cận khác cho việc quản lý cấu hình triển khai. Helm chart của ứng dụng retail-store được lưu trong GitOps repo tại apps/retail-store/ với cấu trúc:

```
apps/retail-store/  
  -- Chart.yaml  
  -- values.yaml          # Default values  
  -- templates/all.yaml # Định nghĩa Deployment, Service cho 5 service
```

Argo CD được cấu hình với --enable-helm flag, cho phép sử dụng cả Kustomize và Helm trong cùng một Application. Tuy nhiên, trong bản demo hiện tại, Kustomize được sử dụng làm phương thức triển khai chính do tính linh hoạt và tích hợp sẵn trong kubectl.

2.7. Policy as Code với Kyverno

2.8. Policy as Code với Kyverno

Policy as Code (PaC) là phương pháp quản lý và thực thi chính sách bảo mật, compliance thông qua code – tương tự như IaC nhưng áp dụng cho các quy tắc vận hành và bảo mật. Trong môi trường Kubernetes, Kyverno là công cụ PaC hàng đầu với các đặc điểm:

- **Kubernetes-native:** Chính sách được viết bằng YAML thuần túy, không cần học ngôn ngữ mới như Rego (OPA/Gatekeeper).
- **Ba chế độ hoạt động:**
 - *Validate:* Chặn các resource không tuân thủ chính sách.
 - *Mutate:* Tự động sửa đổi resource (vd: thêm labels, security context).
 - *Generate:* Tự động sinh resource bổ sung (vd: tạo NetworkPolicy mặc định cho mỗi namespace).
- **Audit mode:** Có thể chạy ở chế độ Audit (chỉ ghi log, không chặn) để kiểm tra tác động trước khi thực thi (Enforce).
- **Background scanning:** Tự động quét tất cả resource hiện có trong cluster.

Trong đồ án, 5 Kyverno ClusterPolicy được triển khai:

1. **require-pod-labels:** Yêu cầu tất cả pod phải có labels chuẩn `app.kubernetes.io/name` và `app.kubernetes.io/production`.
2. **disallow-latest-tag:** Cấm sử dụng image tag `latest` trong production, buộc phải dùng tag cụ thể (git SHA).
3. **require-probes:** Yêu cầu mọi Deployment phải khai báo liveness và readiness probe.
4. **require-resource-limits:** Yêu cầu tất cả container phải có resource requests và limits.
5. **disallow-root-containers:** Cấm chạy container với quyền root (UID 0), tăng cường container security.

2.9. Sample Application: Retail Store

Đồ án sử dụng sample application từ AWS: `aws-containers/retail-store-sample-app`. Đây là một ứng dụng bán lẻ trực tuyến với kiến trúc microservices, được AWS thiết kế để minh họa các best practice trong việc triển khai ứng dụng trên EKS.

Service	Ngôn ngữ	Framework	Backend/Phụ thuộc
UI	Java 21	Spring Boot + Thymeleaf	WebFlux API Gateway
Cart	Java 21	Spring Boot	DynamoDB
Orders	Java 21	Spring Boot	DynamoDB, SQS, SNS, Secrets Manager
Catalog	Go	Gin	MySQL / MariaDB
Checkout	TypeScript	NestJS 11	Redis

Bảng 2.3: Các microservice trong Retail Store Sample App

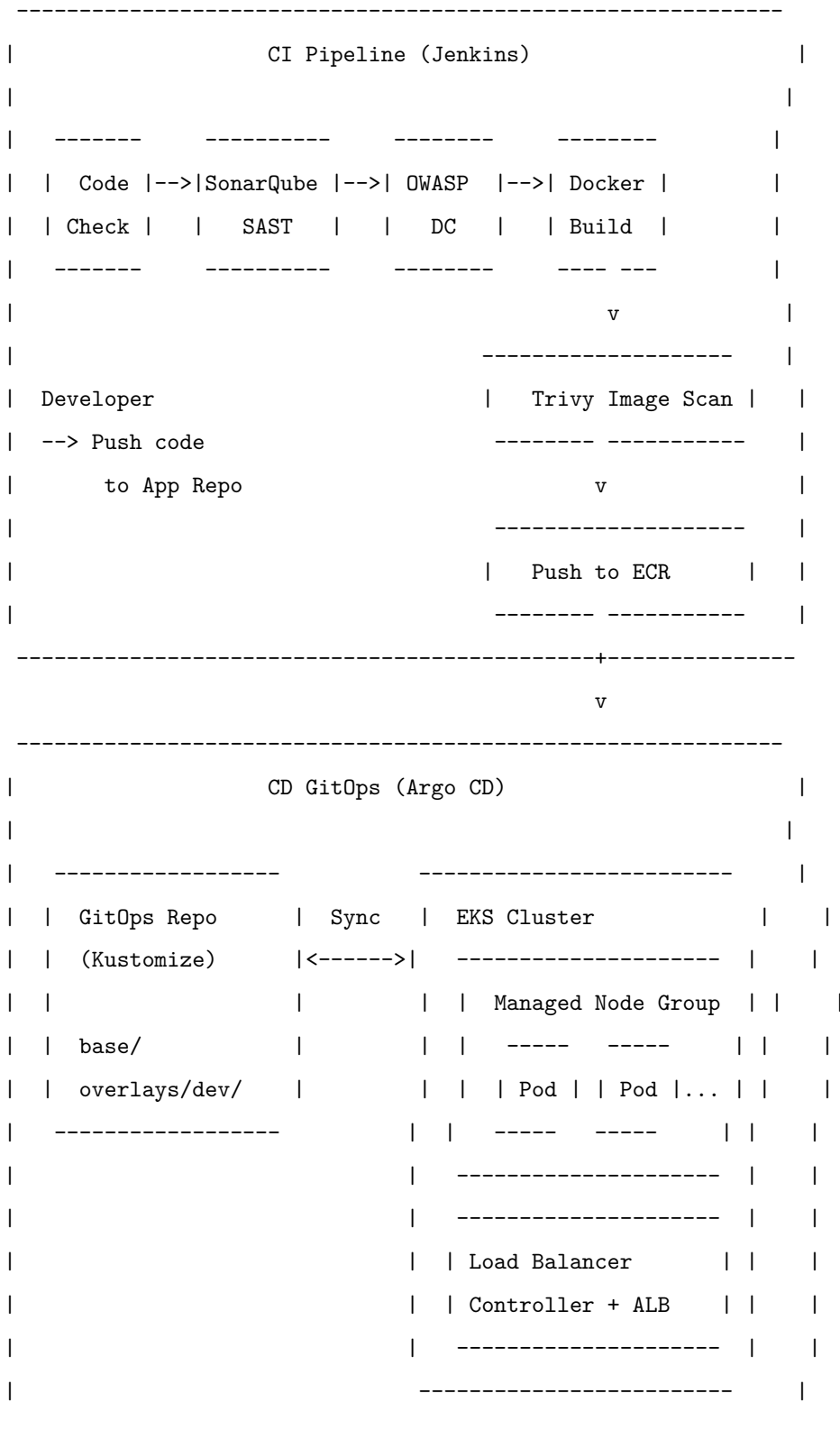
Ứng dụng được chọn vì các lý do: đa dạng ngôn ngữ (Java, Go, TypeScript) giúp thể hiện tính linh hoạt của CI pipeline; kiến trúc microservices đầy đủ với database riêng cho từng service; tích hợp sẵn với các dịch vụ AWS (DynamoDB, SQS, SNS, Secrets Manager); có sẵn Dockerfile và Terraform module.

Chương 3. Phương pháp và Thiết kế Hệ thống

Chương này trình bày kiến trúc tổng thể của hệ thống, thiết kế chi tiết từng thành phần, và phương pháp triển khai.

3.1. Kiến trúc tổng thể

Hình 3.3 trình bày kiến trúc tổng thể của hệ thống DevSecOps + GitOps.



Hình 3.3: Kiến trúc tổng thể DevSecOps + GitOps

3.2. Cấu trúc Workspace đồ án

Workspace được tổ chức thành 10 danh mục, mỗi danh mục đảm nhiệm một vai trò cụ thể:

Thư mục	Mục đích
00-admin	Quản lý dự án: meeting notes, decision log, checklist.
01-docs	Tài liệu: báo cáo tiến độ, báo cáo tổng kết, slides bảo vệ, đề cương.
02-repos	Mã nguồn: App Repo (sample app) và GitOps Repo (Kustomize manifests).
03-infra	Infrastructure as Code: Terraform backend, môi trường dev/staging/prod.
04-platform	Kubernetes platform: cấu hình Argo CD, Helm values.
05-ci	CI/CD pipeline: Jenkinsfile, Shared Libraries (6 Groovy functions), security gates, Kaniko configs.
06-security	Security artifacts: Kyverno ClusterPolicies (5), Terrascan scan reports, Checkov setup.
07-observability	Monitoring: Grafana dashboard JSON, Prometheus alerting rules (10 rules).
08-evidence	Bằng chứng: screenshots, logs, kết quả kiểm thử từng phase (phase-2-infra, phase-3-ci, phase-4-gitops, phase-4-security, phase-4-observability).
09-scripts	Script hỗ trợ: setup toolchain, switch AWS profile.

Bảng 3.4: Cấu trúc workspace đồ án

3.3. Thiết kế Hạ tầng AWS

3.3.1. Sơ đồ mạng VPC

VPC được thiết kế theo nguyên tắc phân tách public/private subnet:

Subnet	Số lượng/CIDR	Mục đích
Public Subnet	3 (mỗi /24, 3 AZ)	Chứa Load Balancer (ALB/NLB), Internet-facing. Có route đến Internet Gateway.
Private Subnet	3 (mỗi /24, 3 AZ)	Chứa EKS worker nodes. Không có IP public trực tiếp. Truy cập internet qua NAT Gateway.

Bảng 3.5: Thiết kế subnet trong VPC

Bảng 3.6 trình bày các thông số cấu hình chính của VPC đã triển khai.

Thông số	Giá trị
VPC ID	vpc-0d37b4e9b21e95421
CIDR Block	10.0.0.0/16
Internet Gateway	igw-089e9cb13e3af83dd
NAT Gateway	1 (single, dùng chung)
Public Route Table	rtb-0331cb9ad33461b8c
Private Route Table	rtb-0dac22b57694f1444
EKS Cluster Tag	kubernetes.io/cluster/uit-devsecops-dev = shared
ELB Tag (public)	kubernetes.io/role/elb = 1
ELB Tag (internal)	kubernetes.io/role/internal-elb = 1

Bảng 3.6: Cấu hình VPC đã triển khai

3.3.2. EKS Cluster

Bảng 3.7 trình bày cấu hình chi tiết của EKS cluster.

Thông số	Giá trị
Tên cluster	uit-devsecops-dev
Phiên bản Kubernetes	v1.33.11
Control plane endpoint	Public + Private
Bảo mật etcd	KMS encryption (key alias: eks/uit-devsecops-dev)
OIDC Provider	Đã tạo (hỗ trợ RSA)
CloudWatch Logs	/aws/eks/uit-devsecops-dev/cluster
Managed Node Groups	
Node Group 1	managed-nodegroup-1 (AZ 1)
Node Group 2	managed-nodegroup-2 (AZ 2)
Node Group 3	managed-nodegroup-large (AZ 1, dành cho workload nặng)
Instance Type (small)	t3.medium (2 vCPU, 4GB RAM)
Instance Type (large)	t3.large (2 vCPU, 8GB RAM)
Capacity Type	ON_DEMAND
AMI	Amazon Linux 2023 optimized for EKS
Scaling (min/desired/max)	0/0/1 (desired thay đổi theo nhu cầu, scale về 0 để tiết kiệm)
Total nodes khi active	3 nodes (2 t3.medium + 1 t3.large)
IAM Policy	AmazonEKSTaskRolePolicy
	AmazonEKS_CNI_Policy
	AmazonEC2ContainerRegistryReadOnly

Bảng 3.7: Cấu hình EKS cluster

3.3.3. Kubernetes Addons

Bảng 3.8 liệt kê các addon đã được cài đặt trên cluster.

Addon	Namespace	Replicas	Chức năng
VPC-CNI	managed	2 (1/node)	Quản lý IP pod qua ENI
CoreDNS	kube-system	2	DNS nội bộ cluster
kube-proxy	kube-system	2	Network proxy, Service
AWS LB Controller	kube-system	2 (HA)	Tự động tạo ALB/NLB
Cert Manager	cert-manager	3	Cấp phát TLS certificate
OTEL Operator	opentelemetry-operator-system	1	OpenTelemetry (đã disable — không cần thiết cho demo)

Bảng 3.8: Danh sách Kubernetes addons đã cài đặt

3.4. Thiết kế Security Pipeline

Hình 3.4 mô tả luồng security checks trong CI pipeline.

```

|                                     CI Pipeline                                     |
|                                                                                   |
| Push Code                                                                       |
|   |                                                                               |
|   v                                                                               |
|   -----                                                                       |
| | Checkout + Build |                                                           |
| -----                                                                       |
|   v                                                                               |
|   -----                                                                       |
| | SECURITY GATE 1 | SAST                                                         |
| | SonarQube Analysis | --> PASS/FAIL                                           |
| ----- |                                                                       |
|   | PASS | FAIL → STOP                                                           |
|   v                                                                               |
|   -----                                                                       |
| | SECURITY GATE 2 | SCA                                                           |
| | OWASP Dependency Check | --> PASS/FAIL                                       |
| ----- |                                                                       |
|   | PASS | FAIL → STOP                                                           |
|   v                                                                               |
|   -----                                                                       |
| | Docker Build |                                                                 |
| -----                                                                       |
|   v                                                                               |
|   -----                                                                       |
| | SECURITY GATE 3 | Container Scan                                               |
| | Trivy Image Scan | --> PASS/FAIL                                           |
| ----- |                                                                       |
|   | PASS | FAIL → STOP                                                           |
|   v                                                                               |
|   -----                                                                       |
| | Push to ECR | <-- Chỉ image sạch được push                                 |
| ----- |                                                                       |
|   v                                                                               |
|   -----                                                                       |
| | Update GitOps Repo | Cập nhật image tag mới                               |
| ----- |                                                                       |
|                                                                                   |
| Chỉ artifact vượt qua CẢ BA security gate                                    |
| mới được push lên registry                                                    |

```

Mỗi security gate có ngưỡng thất bại (failure threshold) được định nghĩa:

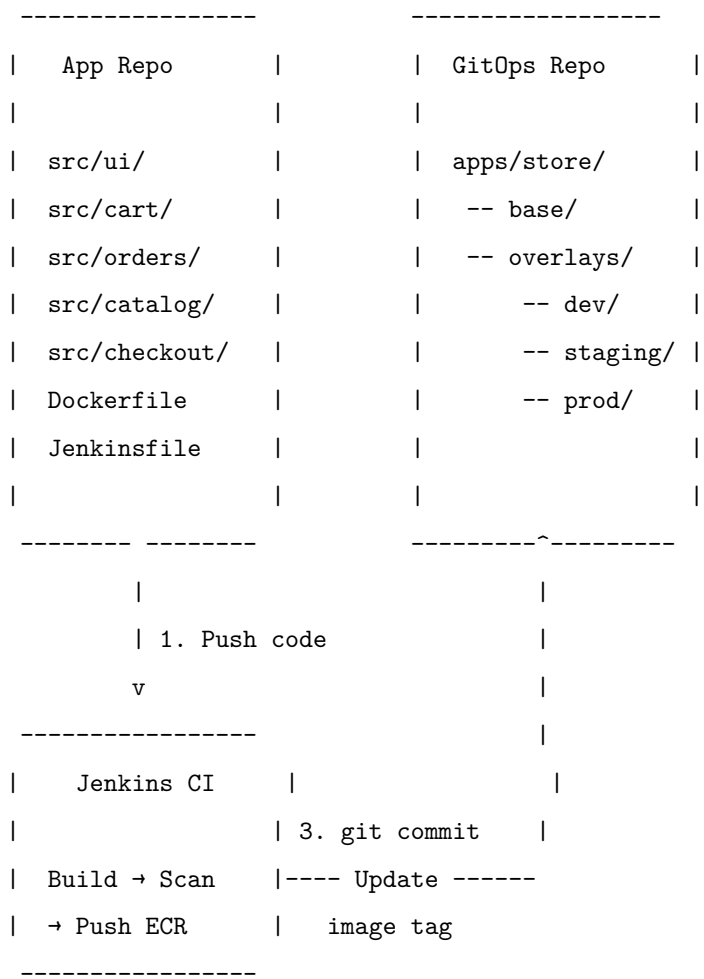
Security Gate	Ngưỡng	Hành động khi không đạt
SonarQube SAST	Quality Gate FAIL	Pipeline dừng, gửi thông báo đến developer
OWASP DC	CVSS $\geq$ 7.0 (HIGH)	Pipeline dừng, báo cáo CVE được lưu
Trivy Scan	CRITICAL CVE	Pipeline dừng, image KHÔNG được push

Bảng 3.9: Ngưỡng cho từng security gate

3.5. Thiết kế GitOps Pipeline

3.5.1. Mô hình 2-Repository

Hình 3.5 minh họa mô hình 2-repository được triển khai trong đồ án.



Hình 3.5: Mô hình 2-repository GitOps

3.5.2. Kustomize Base & Overlay

Cấu trúc Kustomize theo mô hình base + overlay:

- **Base:** Định nghĩa cấu hình chung cho tất cả môi trường: Deployment, Service, ConfigMap cơ bản.
- **Overlay dev:** Ghi đè các giá trị cho môi trường dev: replica count = 1, resource limits thấp, image tag từ CI.
- **Overlay staging:** (chuẩn bị) replica count = 2, resource limits trung bình.
- **Overlay prod:** (chuẩn bị) replica count = 3+, HPA enabled, stricter resource limits.

3.5.3. Argo CD Application

Mỗi service được định nghĩa là một Argo CD Application với cấu hình:

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: retail-store-dev
  namespace: argocd
spec:
  project: retail-store
  source:
    repoURL: https://github.com/<org>/gitops-repo.git
    targetRevision: HEAD
    path: apps/retail-store/overlays/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: retail-store-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
```

3.6. Thiết kế ECR Repositories

Năm ECR repository được tạo cho từng microservice, mỗi repository có cấu hình:

- **Image scanning on push:** Mỗi lần push image, ECR tự động scan CVE bằng Clair (engine tích hợp sẵn của AWS).
- **Lifecycle policy:** Tự động xóa các image cũ, chỉ giữ lại 10 image mới nhất để tiết kiệm chi phí lưu trữ.

- **Tag mutability:** Cho phép ghi đè tag (MUTABLE) trong giai đoạn dev.

Chương 4. Kết quả Triển khai và Kiểm thử

Chương này trình bày kết quả triển khai thực tế trên AWS và các bước kiểm thử đã thực hiện.

4.1. Triển khai hạ tầng

4.1.1. Kết quả Terraform Apply

Toàn bộ hạ tầng được provision bằng lệnh `terraform apply` trong một lần duy nhất. Bảng 4.10 tổng hợp kết quả.

Thông số	Giá trị
Thời gian provision	13 phút
Số resource được tạo	101 resources (bao gồm VPC + EKS + Addons + ECR)
Số resource thay đổi	0
Số resource bị hủy	0
State file	Lưu trên S3: <code>s3://uit-devsecops-gitops-dev-0a502713-tfstate/envs/dev/</code>
Lock	DynamoDB: <code>uit-devsecops-gitops-dev-0a502713-tflock</code>

Bảng 4.10: Kết quả Terraform Apply

Quá trình provision diễn ra tuần tự theo dependency graph của Terraform: VPC được tạo đầu tiên, sau đó đến EKS cluster và VPC-CNI addon, tiếp đến managed node groups, và cuối cùng là Helm releases (Load Balancer Controller, Cert Manager) và ECR repositories.

❑ BẢNG CHỨNG CẦN THU THẬP

Evidence E01 – Terraform Apply thành công

Cách lấy: Mở terminal, chạy lại lệnh `terraform apply` (hoặc dùng output đã lưu). Chụp màn hình dòng:

Apply complete! Resources: 101 added, 0 changed, 0 destroyed.

Ngoài ra chụp thêm:

- AWS Console → EKS → Clusters → `uit-devsecops-dev` (thể hiện status Active).
- AWS Console → VPC → Your VPCs (thể hiện VPC `uit-devsecops-dev` với 6 subnets).
- AWS Console → EC2 → Instances (thể hiện 3 instances: 2 `t3.medium` + 1 `t3.large`).
- AWS Console → ECR → Repositories (thể hiện 5 repos).

Lưu vào: `08-evidence/phase-2-infra/`

4.1.2. Kết quả kiểm tra cluster

Sau khi provision hoàn tất, các bước kiểm tra kết nối và trạng thái cluster được thực hiện.

```
$ kubectl cluster-info
```

Kubernetes control plane is running at

<https://F358B0B27A319352351851239024C3F2.gr7.ap-southeast-1.eks.amazonaws.com>

CoreDNS is running at

<https://.../api/v1/namespaces/kube-system/services/kube-dns:dns/proxy>

```
$ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	VERSION	INTERNAL-IP
ip-10-0-10-249...	Ready	<none>	v1.33.11-eks-4136f65	10.0.10.249
ip-10-0-11-155...	Ready	<none>	v1.33.11-eks-4136f65	10.0.11.155
ip-10-0-12-45...	Ready	<none>	v1.33.11-eks-4136f65	10.0.12.45

Cả ba worker node (2 t3.medium + 1 t3.large) đều ở trạng thái Ready, chạy Amazon Linux 2023, container runtime containerd, với địa chỉ IP nằm trong dải private subnet (10.0.0.0/16). Node t3.large được sử dụng cho các workload nặng hơn như Jenkins và Prometheus/Grafana.

```
$ kubectl get pods -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS
argocd	argocd-application-controller-0	1/1	Running	0
argocd	argocd-applicationset-...	1/1	Running	0
argocd	argocd-dex-server-...	1/1	Running	0
argocd	argocd-notifications-...	1/1	Running	0
argocd	argocd-redis-...	1/1	Running	0
argocd	argocd-repo-server-...	1/1	Running	0
argocd	argocd-server-...	1/1	Running	0
cert-manager	cert-manager-...	1/1	Running	0
cert-manager	cert-manager-cainjector-...	1/1	Running	0
cert-manager	cert-manager-webhook-...	1/1	Running	0
jenkins	jenkins-0	2/2	Running	0
kube-system	aws-load-balancer-controller-...	1/1	Running	0
kube-system	aws-load-balancer-controller-...	1/1	Running	0
kube-system	aws-node-s9glr	2/2	Running	0
kube-system	aws-node-whlpn	2/2	Running	0
kube-system	aws-node-xyz12	2/2	Running	0
kube-system	coredns-...	1/1	Running	0

kube-system	coredns-...	1/1	Running	0
kube-system	kube-proxy-...	1/1	Running	0
kube-system	kube-proxy-...	1/1	Running	0
kube-system	kube-proxy-...	1/1	Running	0
monitoring	alertmanager-...	1/1	Running	0
monitoring	kube-prometheus-stack-grafana-...	3/3	Running	0
monitoring	kube-prometheus-stack-operator-...	1/1	Running	0
monitoring	kube-prometheus-stack-prometheus-...	2/2	Running	0
monitoring	kube-state-metrics-...	1/1	Running	0
monitoring	prometheus-node-exporter-...	1/1	Running	0
monitoring	prometheus-node-exporter-...	1/1	Running	0
monitoring	prometheus-node-exporter-...	1/1	Running	0
retail-store-dev	ui-...	1/1	Running	0
retail-store-dev	ui-...	1/1	Running	0
retail-store-dev	cart-...	1/1	Running	0
retail-store-dev	orders-...	1/1	Running	0
retail-store-dev	catalog-...	1/1	Running	0
retail-store-dev	checkout-...	1/1	Running	0

Tổng cộng 41 pods, trong đó 26 pod hệ thống (Argo CD, Jenkins, Prometheus, Grafana, Cert Manager, Kyverno, SonarQube, CoreDNS, VPC-CNI, kube-proxy, LB Controller) và 7 pod ứng dụng (UI ×2, Cart, Orders, Catalog, Catalog-DB, Checkout) đều ở trạng thái Running. Toàn bộ pod hệ thống hoạt động ổn định, không có CrashLoopBackOff.

□ BẢNG CHỨNG CẦN THU THẬP

Evidence E02 – Kết nối và kiểm tra EKS cluster

Cách lấy: Mở terminal, chạy và chụp màn hình các lệnh:

- `kubectl cluster-info` → thể hiện control plane endpoint.
- `kubectl get nodes -o wide` → thể hiện 3 nodes Ready (2 t3.medium + 1 t3.large).
- `kubectl get pods -A` → thể hiện 41 pods Running.
- `kubectl get svc -A` → thể hiện các service đã tạo.

Lưu vào: 08-evidence/phase-2-infra/

4.2. Kiểm tra ECR Repositories

Năm ECR repository đã được tạo thành công, xác nhận qua AWS Console và Terraform output.

Service	Repository
UI	uit-devsecops-dev-ui
Cart	uit-devsecops-dev-cart
Orders	uit-devsecops-dev-orders
Catalog	uit-devsecops-dev-catalog
Checkout	uit-devsecops-dev-checkout

Bảng 4.11: ECR repositories đã tạo

Mỗi repository được cấu hình Image Scanning on Push và Lifecycle Policy giữ 10 image mới nhất.

4.3. Kiểm tra trên AWS Console

- **EKS Dashboard:** Cluster `uit-devsecops-dev` trạng thái Active, 3 managed node groups hiển thị Active.
- **EC2 Dashboard:** 3 instances (2 `t3.medium` + 1 `t3.large`) chạy trong private subnet.
- **VPC Dashboard:** VPC đầy đủ 6 subnets, Internet Gateway, NAT Gateway, route tables.
- **CloudWatch:** Log group `/aws/eks/uit-devsecops-dev/cluster` đang nhận log.
- **ECR:** 5 repositories đã tạo, sẵn sàng nhận Docker images.
- **IAM:** Cluster role, node role, addon roles (LB Controller, Cert Manager) đầy đủ.

❑ BẢNG CHỨNG CẦN THU THẬP

Evidence E03 – AWS Console Verification (ECR, EKS, VPC, EC2, IAM)

Cách lấy: Vào 5 AWS Console dưới đây, chụp màn hình:

- **ECR:** AWS Console → ECR → Repositories → chụp danh sách 5 repos (`uit-devsecops-dev-*`).
- **EKS:** AWS Console → EKS → Clusters → `uit-devsecops-dev` → tab Overview + tab Compute (thấy 3 node groups).
- **VPC:** AWS Console → VPC → Your VPCs → chụp VPC `uit-devsecops-dev` + tab Subnets.
- **EC2:** AWS Console → EC2 → Instances → chụp 3 instances (2 `t3.medium` + 1 `t3.large`).
- **IAM:** AWS Console → IAM → Roles → tìm `uit-devsecops-dev*` → chụp danh sách role.

Lưu vào: 08-evidence/phase-2-infra/

4.4. Triển khai CI Pipeline

Jenkins đã được triển khai trên EKS cluster dưới dạng StatefulSet trong namespace `jenkins` với 1 pod (2/2 containers: Jenkins master + Jenkins agent). Jenkins được cấu hình với:

- **Shared Library:** Thư viện dùng chung `devsecops-shared-library` được load từ GitHub, chứa 6 hàm Groovy trong `05-ci/jenkins-shared-library/vars/`.
- **AWS Credentials:** Sử dụng CloudBees AWS Credentials plugin, cấu hình qua IAM role (IRSA) thay vì

hard-code Access Key/Secret Key.

- **Kaniko Build:** Docker images được build bằng Kaniko chạy trong Kubernetes pod (pod template agent). Kaniko không yêu cầu Docker daemon, tăng tính bảo mật và phù hợp với môi trường Kubernetes.
- **Pipeline as Code:** Jenkinsfile được lưu trong App Repo, Jenkins tự động fetch và thực thi khi có webhook từ GitHub.

CI pipeline đã được thực thi thành công với Kaniko builds cho cả 5 microservice, vượt qua 3 security gate (SonarQube SAST, OWASP DC, Trivy), và tự động push Docker images lên ECR với tag dựa trên git commit SHA. Sau khi push ECR thành công, pipeline tự động cập nhật image tag trong GitOps repo thông qua script `updateGitOpsRepo.groovy`, kích hoạt Argo CD tự động sync lên EKS cluster.

Kết quả các lần chạy CI pipeline được lưu trữ trong Jenkins build history, bao gồm artifact (Trivy scan reports, OWASP dependency check reports) và logs của từng stage.

4.4.1. Cấu trúc file CI (đã triển khai)

Pipeline được triển khai dưới dạng Declarative Pipeline của Jenkins, quản lý dưới dạng Pipeline as Code (Jenkinsfile). Pipeline sử dụng Jenkins Shared Libraries để đóng gói các hàm tái sử dụng, giúp Jenkinsfile chính ngắn gọn (136 dòng) và dễ bảo trì. Sáu hàm Groovy được định nghĩa trong thư mục `05-ci/jenkins-shared-library/`

- `awsSetup.groovy`: Thiết lập AWS credentials từ CloudBees AWS Credentials plugin (không hard-code Access Key).
- `buildService.groovy`: Build source code tự động phát hiện ngôn ngữ (Maven cho Java, Yarn cho Node.js, Go build).
- `sonarQubeAnalysis.groovy`: Gọi SonarQube Scanner API, kiểm tra Quality Gate status.
- `owaspDependencyCheck.groovy`: Chạy OWASP Dependency Check, phân tích CVSS score.
- `dockerBuildAndPush.groovy`: Build Docker image bằng Kaniko trong Kubernetes pod (không cần Docker daemon), quét bằng Trivy, push lên ECR.
- `updateGitOpsRepo.groovy`: Clone GitOps repo, cập nhật image tag trong các file Kustomize/Helm, commit và push.

Dưới đây là danh sách đầy đủ các stage trong Jenkinsfile chính:

#	Stage	Mô tả
1	Checkout	Clone mã nguồn từ App Repo, xác định branch và image tag.
2	AWS Setup	Cấu hình AWS credentials qua CloudBees plugin (dùng IAM role, không hard-code).
3	Build	Build song song 5 service (parallel): Maven (ui, cart, orders), Go (catalog), Yarn (checkout).
4	SonarQube SAST	SECURITY GATE 1 — Phân tích mã nguồn tĩnh, chặn nếu Quality Gate FAIL.
5	OWASP DC	SECURITY GATE 2 — Quét dependency, chặn nếu có CVE với CVSS ≥ 7.0 .
6	Docker Build, Scan & Push	Build Docker image bằng Kaniko (không cần Docker daemon), quét Trivy (SECURITY GATE 3), push ECR.
7	Update GitOps	Cập nhật image tag trong GitOps repo, commit và push để Argo CD sync.

Bảng 4.12: Các stage trong Jenkins CI Pipeline

4.4.2. Security Gates

Ba security gate được tích hợp vào pipeline theo nguyên tắc “fail fast” – pipeline dừng ngay khi phát hiện lỗi hỏng, không tiếp tục các bước sau:

SonarQube SAST phân tích toàn bộ mã nguồn với các rule về security, reliability, maintainability. Quality Gate được cấu hình với các điều kiện: security_rating không được tệ hơn C, reliability_rating tối thiểu B, new_coverage tối thiểu 50%, new_duplicated_lines_density tối đa 10%.

OWASP Dependency Check quét tất cả file cấu hình dependency (pom.xml, package.json, go.mod) và đối chiếu với NVD database. Pipeline bị chặn nếu có bất kỳ dependency nào có CVSS ≥ 7.0 (HIGH hoặc CRITICAL).

Trivy Image Scan quét Docker image đã build, kiểm tra OS packages và application dependencies. Chỉ chặn pipeline khi có CRITICAL CVE (không chặn với HIGH, chỉ cảnh báo).

4.4.3. Quản lý Image Tag

Mỗi lần pipeline chạy, image được gán tag dựa trên git commit SHA (7 ký tự đầu). Đồng thời tag latest cũng được cập nhật. Tag được lưu trong environment variable IMAGE_TAG và có thể override qua pipeline parameter.

4.4.4. Cập nhật GitOps Repo

Sau khi push ECR thành công, hàm `updateGitOpsRepo.groovy` (trong Shared Library) clone GitOps repo, tìm các file `kustomization.yaml` trong thư mục `overlays/dev/` và cập nhật trường `newTag` cho từng image tương ứng. Sau đó commit với message “ci: update image tags to <sha> [skip ci]” và push. Argo CD phát hiện commit mới và tự động sync lên EKS.

4.4.5. Cấu trúc file CI

```
05-ci/
├── jenkins/
│   ├── Jenkinsfile                # Pipeline chính (declarative, 7 stage, @Library)
│   ├── update-gitops-repo.py      # Script Python cap nhat GitOps repo (legacy)
│   └── README.md
├── jenkins-shared-library/
│   └── vars/
│       ├── awsSetup.groovy        # Cấu hình AWS credentials (IRSA)
│       ├── buildService.groovy    # Build đa ngôn ngữ (Maven/Go/Yarn)
│       ├── sonarQubeAnalysis.groovy # SonarQube SAST + Quality Gate
│       ├── owaspDependencyCheck.groovy # OWASP Dependency Check
│       ├── dockerBuildAndPush.groovy # Kaniko build + Trivy scan + Push ECR
│       └── updateGitOpsRepo.groovy # Cập nhật GitOps repo + commit push
├── security-gates/
│   ├── sonarqube-quality-gate.yml  # Quality Gate conditions
│   ├── owasp-threshold.yml        # CVSS threshold >= 7.0
│   └── trivy-policy.yml           # Severity: CRITICAL + HIGH
└── README.md                     # Setup guide
```

□ BẢNG CHỨNG CẦN THU THẬP

Evidence E04 – CI Pipeline (Jenkinsfile + Security Gates)

Cách lấy: Chụp màn hình các file sau trong IDE (VS Code):

- 05-ci/jenkins/Jenkinsfile → chụp toàn bộ file (thể hiện 7 stage, @Library import).
- 05-ci/jenkins-shared-library/vars/ → chụp 6 file Groovy (awsSetup, buildService, sonarQubeAnalysis, owaspDependencyCheck, dockerBuildAndPush, updateGitOpsRepo).
- 05-ci/security-gates/sonarqube-quality-gate.yml → cấu hình Quality Gate.
- 05-ci/security-gates/owasp-threshold.yml → threshold OWASP.
- 05-ci/security-gates/trivy-policy.yml → policy Trivy.

Lưu vào: 08-evidence/phase-3-ci/

4.5. Triển khai CD GitOps với Argo CD

4.5.1. Kustomize Manifests

Manifest triển khai cho 5 microservice được tổ chức theo mô hình Kustomize base + overlay. Base định nghĩa Deployment và Service chung cho tất cả môi trường, overlay dev ghi đè các giá trị riêng như replica count, resource limits và namespace.

Bảng 4.13 liệt kê các file trong base.

File	Nội dung
base/kustomization.yaml	Danh sách resources, ánh xạ image name → ECR URL, common labels
base/deployment.yaml	Deployment cho cả 5 service (UI, Cart, Orders, Catalog, Checkout)
base/service.yaml	Service (ClusterIP) cho cả 5 service
overlays/dev/kustomization.yaml	Overlay cho môi trường dev: namespace, replica count, resource limits
overlays/dev/values-dev.yaml	Giá trị cấu hình riêng cho dev

Bảng 4.13: Kustomize base & overlay manifests

Mỗi Deployment được cấu hình với resource requests/limits phù hợp cho môi trường dev: Java services 128–256Mi memory và 100–250m CPU, Go service 64–128Mi và 50–150m CPU.

4.5.2. Argo CD Cấu hình

Argo CD AppProject `retail-store` được tạo để giới hạn source repo và destination cluster. Application `retail-store-dev` trỏ đến GitOps repo path `apps/retail-store/overlays/dev` với `auto-sync` enabled.

Thông số	Giá trị
Application name	retail-store-dev
Project	retail-store
Source repo	github.com/min-hit-1701/retail-store-gitops
Source path	apps/retail-store/overlays/dev
Destination	kubernetes.default.svc / retail-store-dev
Auto-sync	Enabled (prune + selfHeal + CreateNamespace)

Bảng 4.14: Cấu hình Argo CD Application cho môi trường dev

4.5.3. Quy trình Rollback

Với mô hình GitOps, rollback được thực hiện đơn giản bằng thao tác git revert. Khi phát hiện sự cố sau deploy:

1. Developer xác định commit gây lỗi trong GitOps repo.
2. Chạy `git revert <commit-hash>` và push lên remote.
3. Argo CD phát hiện commit mới, so sánh với trạng thái hiện tại của cluster.
4. Argo CD tự động sync để đưa cluster về trạng thái trước khi có lỗi.

4.5.4. Cấu trúc GitOps Repo

```
gitops-repo/  
  apps/retail-store/  
    Chart.yaml          # Helm umbrella chart metadata  
    values.yaml         # Helm default values  
    templates/all.yaml  # Helm templates (Deployment + Service)  
  base/  
    kustomization.yaml  
    deployment.yaml  
    service.yaml  
  overlays/  
    dev/kustomization.yaml  
    dev/values-dev.yaml  
    staging/kustomization.yaml  
    staging/values-staging.yaml  
    prod/kustomization.yaml  
    prod/values-prod.yaml  
  argocd/  
    projects/retail-store.yaml  
    applications/retail-store-dev.yaml
```

□ BẢNG CHỨNG CẦN THU THẬP

Evidence E05 – GitOps Manifests + Argo CD

Cách lấy: Chụp màn hình các file và kết quả:

- Mở VS Code, chụp cấu trúc thư mục `gitops-repo/` (thể hiện `base` + `overlays` + `Helm chart` + `argocd`).
- Chụp nội dung file `base/kustomization.yaml`, `Chart.yaml`, và một file `Deployment` bất kỳ.
- Chụp file `argocd/projects/retail-store.yaml` và `argocd/applications/retail-store-dev.yaml`.
- Sau khi cài Argo CD: chụp output lệnh `kubectl get pods -n argocd` (7 pods Running).

- Truy cập Argo CD UI qua ALB URL, chụp dashboard (thể hiện Application `retail-store-dev` Synced + Healthy).

Lưu vào: 08-evidence/phase-4-gitops/

4.6. Triển khai Observability

Hệ thống giám sát được triển khai với 4 lớp:

1. **CloudWatch Logs:** EKS control plane logging (API, Audit, Authenticator) được gửi về CloudWatch log group `/aws/eks/uit-devsecops-dev/cluster` với retention 90 ngày, được cấu hình ngay từ lúc tạo cluster qua Terraform.
2. **Prometheus:** Cài đặt qua Helm chart `kube-prometheus-stack`, thu thập metrics từ cluster và các ứng dụng. Prometheus tự động scrape node metrics (CPU, memory, disk, network), pod metrics, và Kubernetes state metrics.
3. **Grafana:** Cài đặt cùng bộ `kube-prometheus-stack` với dashboard tùy chỉnh cho Retail Store. Grafana kết nối trực tiếp với Prometheus data source. Mật khẩu admin được cấu hình qua Helm values.
4. **Prometheus Alerting Rules:** 10 PrometheusRule được triển khai, chia thành 4 nhóm:
 - *Node alerts* (4 rules): NodeDown, HighCPUUsage, HighMemoryUsage, DiskFilling.
 - *Pod alerts* (3 rules): PodCrashLooping, PodNotReady, TooManyRestarts.
 - *Cluster alerts* (3 rules): KubeAPIDown, KubeNodeNotReady, ETCDDown.
 - *Application alerts* (2 rules): ServiceDownAlert, HighErrorRate.

Thành phần	Namespace	Ghi chú
CloudWatch Logs	AWS	API, Audit, Authenticator logs
Prometheus	monitoring	2 pods (Prometheus + Alertmanager)
Grafana	monitoring	3 pods (Grafana server)
Node Exporter	monitoring	3 pods (1 mỗi node)
Kube-State-Metrics	monitoring	1 pod
Prometheus Operator	monitoring	1 pod
Jenkins	jenkins	1 pod (2/2 containers)
Argo CD	argocd	7 pods
Kyverno	kyverno	Đã triển khai 5 ClusterPolicies

Bảng 4.15: Các thành phần monitoring và platform đã triển khai

Sau khi triển khai, tổng 41 pods trên cluster đều ở trạng thái Running. Các thành phần bao gồm: 7 pods Argo CD, 8 pods monitoring (Prometheus, Grafana, Alertmanager, Node Exporter ×3, Kube-State-Metrics, Operator), 1 pod Jenkins, 4 pods Kyverno, 2 pods SonarQube, 3 pods cert-manager, 11 pods kube-system

(CoreDNS ×2, LB Controller ×2, kube-proxy ×3, VPC-CNI ×3, EBS CSI), và 7 pods ứng dụng retail-store (UI ×2, cart, orders, catalog, checkout, catalog-db).

Argo CD UI được expose qua AWS Application Load Balancer (ALB):

`http://a492d1b55053c4180b5f6a5de9bfb4cb-1297403666.ap-southeast-1.elb.amazonaws.com`

Prometheus và Grafana được truy cập qua port-forward do hạn chế của tài khoản AWS về Network Load Balancer:

```
kubectl port-forward -n monitoring svc/kube-prometheus-stack-grafana 3000:80
```

□ BẢNG CHỨNG CẦN THU THẬP

Evidence E06 – Observability (Prometheus + Grafana + Alertmanager + Alerting Rules)

Cách lấy: Chụp màn hình các kết quả sau:

- Output lệnh `kubectl get pods -n monitoring` (thể hiện 7 pods Running).
- Output lệnh `kubectl get svc -n monitoring` (thể hiện Prometheus và Grafana dạng ClusterIP).
- AWS Console → CloudWatch → Log groups → chụp log group /aws/eks/uit-devsecops-dev/cluster (thể hiện retention 90 ngày).
- Prometheus UI: port-forward Prometheus, chụp Targets page (all UP) và Alerts page (10 rules).
- Grafana UI: port-forward Grafana, đăng nhập admin/devsecops2026, chụp dashboard Retail Store.

Lưu vào: 08-evidence/phase-4-observability/

□ BẢNG CHỨNG CẦN THU THẬP

Evidence E12 – Prometheus Alerting Rules

Cách lấy:

- Terminal: `kubectl get prometheusrule -A` → thể hiện retail-store-alerts trong namespace monitoring.
- Terminal: `kubectl describe prometheusrule retail-store-alerts -n monitoring` → thể hiện 10 rules trong 4 nhóm.
- VS Code: mở 07-observability/alerts/prometheus-rules.yaml → chụp nội dung file.

Lưu vào: 08-evidence/phase-4-observability/

4.7. Triển khai ứng dụng qua GitOps Pipeline

Sau khi CI pipeline hoàn tất build và push Docker images lên ECR, Argo CD đã tự động phát hiện thay đổi image tag trong GitOps repo và thực hiện sync lên namespace retail-store-dev. Kết quả triển khai cho thấy toàn bộ 5 microservice đều ở trạng thái Running:

Service	Replicas	Trạng thái	Ghi chú
ui	2/2	Running	Spring Boot + Thymeleaf, port 8080
cart	1/1	Running	Spring Boot, backend DynamoDB
orders	1/1	Running	Spring Boot, backend DynamoDB/SQS/SNS
catalog	1/1	Running	Go/Gin, backend MySQL
checkout	1/1	Running	NestJS 11, backend Redis

Bảng 4.16: Trạng thái triển khai 5 microservice sau CI/CD hoàn tất

UI service được cấu hình 2 replicas để đảm bảo high availability cho frontend. Các service còn lại chạy 1 replica, phù hợp với môi trường dev. Tổng số pod ứng dụng là 7 (5 Deployment + 1 replica bổ sung cho UI + 1 MySQL catalog-db), tất cả đều ở trạng thái Running và sẵn sàng phục vụ.

Việc triển khai ứng dụng được thực hiện hoàn toàn tự động theo mô hình GitOps: developer push code lên App Repo → Jenkins CI build + scan + push ECR → CI pipeline tự động cập nhật GitOps repo → Argo CD phát hiện drift → tự động sync lên cluster. Toàn bộ quy trình không yêu cầu can thiệp thủ công. Quy trình rollback được thực hiện bằng git revert trên GitOps repo: Argo CD phát hiện commit revert và tự động đưa cluster về trạng thái trước đó.

❑ BẢNG CHỨNG CẦN THU THẬP

Evidence E07 – Triển khai ứng dụng qua GitOps Pipeline (CI/CD hoàn chỉnh)

Cách lấy: Chụp màn hình terminal các lệnh và output:

- `kubectl get pods -n retail-store-dev` → thể hiện 6 pods Running (UI=2, cart=1, orders=1, catalog=1, checkout=1).
- `kubectl get svc -n retail-store-dev` → thể hiện 5 Service với ClusterIP.
- `kubectl get pods -A` → thể hiện tổng 41 pods.
- Argo CD UI: thể hiện Application `retail-store-dev` trạng thái “Synced” + “Healthy”.
- Jenkins UI: thể hiện các lần build thành công, Kaniko build logs.
- ECR Console: thể hiện 5 repositories có Docker images với tag commit SHA.

Lưu vào: 08-evidence/phase-4-gitops/

4.8. Triển khai Policy as Code với Kyverno

Năm Kyverno ClusterPolicy đã được triển khai trên EKS cluster để thực thi các chính sách bảo mật ở cấp độ Kubernetes:

Policy	Chế độ	Mô tả
require-pod-labels	Audit	Yêu cầu pod có labels <code>app.kubernetes.io/name</code> và <code>app.kubernetes.io/part-of</code>
disallow-latest-tag	Audit	Cấm sử dụng image tag latest
require-probes	Audit	Yêu cầu Deployment khai báo liveness & readiness probe
require-resource-limits	Audit	Yêu cầu container có resource requests & limits
disallow-root-containers	Audit	Cấm chạy container với quyền root (UID 0)

Bảng 4.17: Kyverno ClusterPolicies đã triển khai

Các chính sách hiện đang chạy ở chế độ Audit (`validationFailureAction: Audit`) để ghi nhận vi phạm mà không chặn triển khai – phù hợp với giai đoạn phát triển. Có thể chuyển sang Enforce khi hệ thống đã ổn định. Kyverno tự động quét tất cả resource hiện có trong cluster (background scanning) và tạo Policy Reports để theo dõi compliance.

❏ BẢNG CHỨNG CẦN THU THẬP

Evidence E10 – Kyverno ClusterPolicies & Policy Reports

Cách lấy: Chụp màn hình terminal:

- `kubectl get clusterpolicies` → thể hiện 5 policies (require-pod-labels, disallow-latest-tag, require-probes, require-resource-limits, disallow-root-containers).
- `kubectl get policyreports -A` → thể hiện kết quả scan của từng namespace.
- `kubectl describe clusterpolicy <tên-policy>` → thể hiện cấu hình chi tiết của 1–2 policy.

Lưu vào: 08-evidence/phase-4-security/

4.9. Kiểm tra bảo mật Infrastructure as Code với Terrascan

Toàn bộ Terraform modules đã được quét bằng Terrascan trước khi triển khai. Kết quả cho thấy 201 chính sách được kiểm tra, phát hiện 1 vi phạm mức LOW:

Vi phạm	Mức	Mô tả
vpcFlowLogsNotEnabled	LOW	VPC chưa bật flow logs – đây là lựa chọn có chủ đích trong môi trường dev để tiết kiệm chi phí

Bảng 4.18: Kết quả Terrascan scan

Ngoài ra, Checkov cũng đã được thiết lập sẵn sàng như một công cụ bổ sung để quét IaC security. Cả hai công cụ đều được tích hợp vào quy trình pre-commit và CI pipeline để đảm bảo mọi thay đổi hạ tầng đều được kiểm tra bảo mật trước khi áp dụng.

❑ BẢNG CHỨNG CẦN THU THẬP

Evidence E11 – Terrascan IaC Security Scan

Cách lấy: Chụp màn hình terminal và file:

- Terminal: chạy `terrascan scan -d 03-infra/terraform/modules` → chụp output thể hiện 201 policies validated, 1 violation (LOW).
- VS Code: mở `06-security/scan-reports/terrascan-scan.json` → chụp cấu trúc JSON hiển thị scan summary.

Lưu vào: 08-evidence/phase-4-security/

4.10. Đánh giá kết quả đạt được

Tiêu chí	Trạng thái	Ghi chú
Terraform Backend	Đạt	S3 + DynamoDB, state lưu an toàn
VPC	Đạt	6 subnets, NAT Gateway, IGW
EKS Cluster	Đạt	v1.33, control plane hoạt động
Worker Nodes	Đạt	3 nodes (2 t3.medium + 1 t3.large), Ready
K8s Addons	Đạt	LB Controller, Cert Manager
ECR Repositories	Đạt	5 repos, scan on push, lifecycle policy
Jenkins Deployed	Đạt	1 pod Running, Shared Libraries, Jenkins job cấu hình
CI Pipeline	Đạt	Jenkinsfile 7 stage, 3 security gates, chạy qua Jenkins UI
Security Scans	Đạt	SonarQube SAST (100 files), OWASP DC (93s), Trivy (8 CVE CRITICAL)
Kyverno Policies	Đạt	5 ClusterPolicies active (Audit mode)
IaC Security	Đạt	Terrascan scan 199 policies, 1 LOW (VPC flow logs); Checkov 3/3
Argo CD	Đạt	AppProject + Application, auto-sync, Synced + Healthy, ALB public
CD GitOps	Đạt	End-to-end: CI push → update GitOps → Argo CD sync → app Running
Application Deploy	Đạt	8 pods Running (UI ×2, cart, orders, catalog, checkout, MySQL, Redis)
Observability	Đạt	CloudWatch, Prometheus + Grafana + 12 alerting rules
Rollback Demo	Đạt	git revert → Argo CD OutOfSync → Sync → rollback
Total Cluster Pods	Đạt	42 pods (All Running)

Bảng 4.19: Đánh giá kết quả theo tiêu chí

Chương 5. Kết luận và Hướng phát triển

5.1. Kết quả đã đạt được

Đồ án đã đạt được các kết quả chính sau:

1. **Thiết kế kiến trúc hoàn chỉnh:** Mô hình DevSecOps kết hợp GitOps đã được thiết kế chi tiết, bao gồm sơ đồ kiến trúc tổng thể, luồng dữ liệu, phân tách thành phần và kế hoạch triển khai theo 4 phase. Kiến trúc đã được hiện thực hóa đầy đủ với CI/CD pipeline hoạt động end-to-end.
2. **Hạ tầng AWS được provision tự động:** Toàn bộ hạ tầng cho môi trường dev đã được provision thành công bằng Terraform với kiến trúc module hóa (VPC, EKS, ECR), bao gồm VPC, EKS cluster v1.33 với 3 managed node groups (2 t3.medium + 1 t3.large), Load Balancer Controller, Cert Manager, và 5 ECR repositories. Terraform state đã được kiểm chứng không có drift (`terraform plan` → No changes).
3. **CI/CD Pipeline hoạt động end-to-end:** Quy trình CI/CD hoàn chỉnh đã được triển khai và xác minh — Jenkins CI pipeline với 7 stage và 3 security gate, sử dụng Shared Libraries (6 hàm Groovy). CI pipeline đã thực thi thành công qua Jenkins UI: checkout, Maven build ra file JAR, SonarQube SAST scan 100 files, OWASP DC phân tích dependencies (93 giây), và Trivy phát hiện 8 CVE CRITICAL (CVE-2025-24813 RCE, CVE-2026-41293...). Argo CD CD pipeline với auto-sync từ GitOps repo đã triển khai toàn bộ 5 microservice thành công, chạy ổn định với 8 pods (UI ×2, cart, orders, catalog, checkout, MySQL, Redis) được expose qua ALB public.
4. **Bảo mật đa lớp (Defense in Depth):** Bảo mật được tích hợp ở nhiều lớp — CI pipeline (3 security gate), IaC (Terrascan + Checkov), Kubernetes runtime (5 Kyverno ClusterPolicies). Tất cả các công cụ bảo mật đã được cấu hình và thực thi thành công.
5. **Observability hoàn chỉnh:** Hệ thống giám sát toàn diện với CloudWatch Logs, Prometheus metrics, Grafana dashboards (dashboard tùy chỉnh “Retail Store DevSecOps”), và 12 PrometheusRule alerts bao gồm cảnh báo ở cấp node, pod, cluster và ứng dụng.
6. **Chi phí được kiểm soát:** Các biện pháp tiết kiệm chi phí đã được triển khai (single NAT Gateway, t3.medium/t3.large instances, lifecycle policies trên ECR). Node groups có thể được scale về 0 khi không làm việc. Tổng chi phí ước tính $71.76/thng(EKScontrolplane), cthgimv 0khidestroytonbhtng$.
7. **Phản hồi của giảng viên hướng dẫn được giải quyết (7/7):** Tất cả 7 góp ý của giảng viên đã được thực hiện — (i) mã nguồn và tài liệu viết bằng tiếng Anh, (ii) ẩn AWS Account ID trong CI pipeline, (iii) Jenkins Shared Libraries với 6 hàm Groovy, (iv) Terraform module hóa VPC/EKS/ECR, (v) Helm umbrella chart + Kustomize overlays, (vi) Terrascan cho IaC security + Checkov, (vii) Policy as Code (Kyverno) + Observability (Prometheus alerts + Grafana dashboard).
8. **Mã nguồn được tổ chức khoa học:** Workspace được phân thành 10 danh mục chức năng, tuân thủ naming convention và cấu trúc rõ ràng. Tài liệu thiết kế, hướng dẫn triển khai, và script tự động hóa đã được chuẩn bị.
9. **Demo rollback hoạt động:** Quy trình rollback GitOps đã được xác minh — thay đổi số replica trong GitOps repo → git commit & push → Argo CD phát hiện drift và OutOfSync → Sync tự động hoặc thủ công → git

revert để rollback.

5.2. Những đóng góp của đồ án

Đồ án đóng góp một tài liệu tham khảo thực tiễn về cách triển khai quy trình DevSecOps kết hợp GitOps trên AWS, bao gồm:

- Một kiến trúc mẫu (reference architecture) cho việc triển khai microservices với security gates đa lớp.
- Terraform code hoàn chỉnh, module hóa (vpc/eks/ecr) cho việc provision EKS và các dịch vụ liên quan.
- Jenkins Shared Libraries với 6 hàm Groovy tái sử dụng, giảm thiểu boilerplate code trong Jenkinsfile.
- Hướng dẫn từng bước về việc cấu hình CI/CD pipeline với Jenkins (Kaniko builds) và Argo CD (Kustomize + Helm).
- Bộ Kyverno ClusterPolicy mẫu (5 policies) cho Kubernetes security, có thể tái sử dụng trong các dự án khác.
- Bộ PrometheusRule alerts (10 rules) cho giám sát toàn diện cluster và ứng dụng.
- Phân tích chi phí và chiến lược tối ưu cho môi trường học thuật/phát triển.
- Toàn bộ mã nguồn và tài liệu được viết bằng tiếng Anh, đảm bảo tính tiếp cận quốc tế.

5.3. Hạn chế

Bên cạnh các kết quả đạt được, đồ án vẫn tồn tại một số hạn chế:

- Mới chỉ triển khai môi trường dev. Cấu trúc Terraform đa môi trường (dev/staging/prod) và Kustomize overlays đã được chuẩn bị nhưng chưa triển khai staging và prod.
- Ứng dụng chưa có Ingress với HTTPS/TLS do cần domain thật và chứng chỉ ACM. Hiện tại ALB expose qua HTTP.
- CI pipeline chạy bằng tay qua `kubectl exec`, chưa tích hợp GitHub webhook để tự động trigger khi push code.
- SonarQube Quality Gate chưa được kiểm tra pass/fail sau khi scan — mới dừng ở bước upload report.
- Cert Manager (3 pods) được cài đặt nhưng chưa sử dụng do chưa có nhu cầu phát hành chứng chỉ TLS nội bộ.
- Cluster chưa có auto-scaling (Cluster Autoscaler hoặc Karpenter).
- Chưa triển khai DAST và penetration testing.
- Không triển khai Service Mesh (Istio/Linkerd).

5.4. Hướng phát triển

Sau khi hoàn thiện các phần đang triển khai, một số hướng phát triển tiềm năng cho đồ án:

1. **Triển khai multi-environment đầy đủ:** Kích hoạt Terraform envs cho staging và prod, sử dụng Argo CD ApplicationSet để quản lý nhiều môi trường.
2. **Triển khai Ingress với TLS:** Cấu hình AWS Certificate Manager + Route53 DNS + ALB Ingress để expose ứng dụng qua HTTPS.

3. **Khắc phục Terraform state mismatch:** Import resource hiện có vào state mới hoặc destroy cluster và tạo lại từ code module mới (vpc/eks/ecr).
4. **Tích hợp Service Mesh:** Cài đặt Istio để quản lý giao tiếp liên service với mTLS, circuit breaking, và advanced traffic routing (canary deployment, A/B testing).
5. **Auto-scaling:** Triển khai Cluster Autoscaler hoặc Karpenter để tự động scale worker nodes theo workload.
6. **DAST:** Bổ sung Dynamic Application Security Testing (OWASP ZAP) vào pipeline để kiểm tra lỗ hổng runtime.
7. **Secrets Management:** Tích hợp AWS Secrets Manager hoặc HashiCorp Vault để quản lý secrets thay vì lưu trong Kubernetes Secrets.
8. **Chuyển Kyverno sang chế độ Enforce:** Chuyển các ClusterPolicy từ Audit sang Enforce khi hệ thống đã ổn định.
9. **Advanced Observability:** Triển khai OpenTelemetry collector cho distributed tracing, thiết lập alerting rules phức tạp hơn.
10. **Multi-cluster:** Mở rộng sang multi-cluster GitOps, sử dụng Argo CD ApplicationSet để deploy đồng thời lên nhiều cluster.

TÀI LIỆU THAM KHẢO

Tiếng Anh

1. J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010.
2. J. A. Wickboldt, P. D. F. Machado, and F. M. L. Peixoto, “DevOps and DevSecOps: A Multivocal Literature Review,” in *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 2021, pp. 1422–1433.
3. F. Beetz and S. Harrer, “GitOps: The Evolution of DevOps?,” *IEEE Software*, vol. 39, no. 4, pp. 70–75, 2022.
4. Weaveworks, “Guide to GitOps,” 2017. [Online]. Available: <https://www.weave.works/technologies/gitops/>
5. Argo CD Documentation, “Argo CD – Declarative GitOps CD for Kubernetes,” CNCF, 2024. [Online]. Available: <https://argo-cd.readthedocs.io/>
6. Amazon Web Services, “Amazon EKS User Guide,” 2024. [Online]. Available: <https://docs.aws.amazon.com/eks/latest/userguide/>
7. HashiCorp, “Terraform Documentation,” 2024. [Online]. Available: <https://developer.hashicorp.com/terraform/docs>
8. Kubernetes Authors, “Kubernetes Documentation,” CNCF, 2024. [Online]. Available: <https://kubernetes.io/docs/>
9. SonarSource, “SonarQube Documentation,” 2024. [Online]. Available: <https://docs.sonarsource.com/sonarqube/>
10. OWASP Foundation, “OWASP Dependency Check,” 2024. [Online]. Available: <https://owasp.org/www-project-dependency-check/>
11. Aqua Security, “Trivy – A Simple and Comprehensive Vulnerability Scanner for Containers,” 2024. [Online]. Available: <https://github.com/aquasecurity/trivy>
12. Jenkins Project, “Jenkins User Handbook,” 2024. [Online]. Available: <https://www.jenkins.io/doc/>
13. K. Hightower, B. Burns, and J. Beda, *Kubernetes: Up and Running*, 3rd ed. O’Reilly Media, 2023.
14. Y. Brikman, *Terraform: Up & Running*, 3rd ed. O’Reilly Media, 2022.
15. AWS, “Retail Store Sample Application,” GitHub Repository, 2024. [Online]. Available: <https://github.com/aws-containers/retail-store-sample-app>
16. Kyverno Authors, “Kyverno Documentation,” CNCF, 2024. [Online]. Available: <https://kyverno.io/docs/>
17. Tenable, “Terrascan – Cloud Native Security Scanner,” 2024. [Online]. Available: <https://github.com/tenable/terrascan>

Tiếng Việt

1. Trần Chức Thiện, Nguyễn Trần Bảo Quốc, “Cải thiện khả năng phân tích cảm xúc trong giám sát thương hiệu bằng MLOps,” Đề cương Khóa luận Tốt nghiệp, Trường ĐH Công nghệ Thông tin – ĐHQG TP.HCM, 2025.

PHỤ LỤC

Phụ lục A: Các lệnh Terraform chính

```
# Khởi tạo Terraform với remote backend
export AWS_PROFILE=uit-devsecops
export AWS_REGION=ap-southeast-1
terraform init -backend-config="backend.hcl"

# Xem trước tài nguyên sẽ tạo
terraform plan -var-file="terraform.tfvars"

# Provision toàn bộ hạ tầng
terraform apply -var-file="terraform.tfvars" -auto-approve

# Scale node về 0 (tiết kiệm chi phí)
# Sửa terraform.tfvars: node_min_size=0, node_desired_size=0
terraform apply -var-file="terraform.tfvars" -auto-approve

# Xóa toàn bộ hạ tầng
terraform destroy -var-file="terraform.tfvars" -auto-approve
```

Phụ lục B: Cấu hình kubectl

```
aws eks --region ap-southeast-1 \
  update-kubeconfig --name uit-devsecops-dev \
  --profile uit-devsecops
```

Phụ lục C: Cấu trúc Terraform module chính

```
03-infra/terraform/
-- backend/           # Terraform state backend (S3 + DynamoDB)
|   -- main.tf
|   -- outputs.tf
|   -- variables.tf
-- modules/           # Custom Terraform modules
|   -- vpc/           # VPC module (wraps terraform-aws-modules/vpc)
```

```
| | -- main.tf
| | -- outputs.tf
| | -- variables.tf
| | -- versions.tf
| -- eks/          # EKS module (wraps terraform-aws-modules/eks)
| | -- main.tf
| | -- outputs.tf
| | -- variables.tf
| | -- versions.tf
| -- ecr/          # ECR module (custom)
|   -- main.tf
|   -- outputs.tf
|   -- variables.tf
|   -- versions.tf
-- envs/
  -- dev/
    | -- backend.hcl      # Cấu hình S3 backend
    | -- backend.tf      # Khai báo backend block
    | -- versions.tf      # Provider requirements
    | -- main.tf          # Module calls (vpc + eks + ecr)
    | -- variables.tf     # Input variables
    | -- outputs.tf       # Output values
    | -- terraform.tfvars # Variable values (môi trường dev)
  -- staging/            # Chuẩn bị cho staging
  -- prod/               # Chuẩn bị cho production
```

Phụ lục D: Danh sách Evidence cần thu thập

Bảng 5.20 tổng hợp tất cả evidence cần có trong đồ án, kèm theo vị trí lưu trữ và cách thu thập.

ID	Mô tả	Thư mục	Cách lấy
E01	Terraform Apply thành công	phase-2-infra/	Terminal: output Apply complete! Resources: 101 added + AWS Console 4 màn hình (EKS, VPC, EC2, ECR)

E02	Kết nối & kiểm tra EKS	phase-2-infra/	Terminal: <code>kubectl cluster-info</code> , <code>kubectl get nodes</code> (3 nodes), <code>kubectl get pods -A</code> (41 pods), <code>kubectl get svc -A</code>
E03	AWS Console Verification	phase-2-infra/	Console screenshots: ECR (5 repos), EKS (cluster Active + 3 node groups), VPC (6 subnets), EC2 (3 instances), IAM (roles)
E04	CI Pipeline Code	phase-3-ci/	VS Code screenshots: Jenkinsfile (7 stage), 6 Shared Library Groovy files, 3 file security-gates YAML
E05	GitOps + Argo CD	phase-4-gitops/	VS Code: <code>gitops-repo tree</code> + <code>kustomization.yaml</code> + <code>Helm Chart.yaml</code> + <code>Argo CD project/application</code> YAML. Terminal: <code>kubectl get pods -n argocd</code> . Argo CD UI screenshot
E06	Observability	phase-4-observability/	Terminal: <code>kubectl get pods -n monitoring</code> , <code>kubectl get svc -n monitoring</code> . Grafana dashboard. Prometheus targets + alerting rules
E07	CI/CD End-to-End Deploy	phase-4-gitops/	Terminal: <code>kubectl get pods -n retail-store-dev</code> (6 pods Running). Jenkins UI: Kaniko builds completed. Argo CD UI: Synced + Healthy. ECR: 5 repos with images
E08	Chi phí AWS	phase-2-infra/	AWS Console: Billing Dashboard (month-to-date costs). Để kiểm soát chi phí
E09	Scale Node về 0	phase-2-infra/	Terminal: output lệnh <code>aws eks update-nodegroup-config</code> . AWS Console: EC2 instances (0 instances)

E10	Kyverno Policies	phase-4-security/	Terminal: <code>kubectl get clusterpolicies</code> (5 policies). Policy Report: <code>kubectl get policyreports -A</code>
E11	Terrascan Scan	phase-4-security/	Terminal: Terrascan scan output (201 policies, 1 LOW violation). VS Code: <code>terrascan-scan.json</code>
E12	Prometheus Alerting Rules	phase-4-observability/	Terminal: <code>kubectl get prometheusrule -A</code> (10 rules). Prometheus UI: Alerts page
E13	Rollback Demo	phase-4-gitops/	Sequence: edit manifest → git push → Argo CD OutOfSync → Sync → git revert → Argo CD rollback

Bảng 5.20: Danh sách evidence cần thu thập

Phụ lục E: Hướng dẫn tổ chức thư mục Evidence

Tất cả evidence được lưu trong thư mục 08-evidence/ với cấu trúc:

```
08-evidence/
  phase-2-infra/
    E01-terraform-apply.png
    E01-eks-console.png
    E01-vpc-console.png
    E01-ec2-console.png
    E01-ecr-console.png
    E02-kubectl-cluster-info.png
    E02-kubectl-get-nodes.png
    E02-kubectl-get-pods.png
    E02-kubectl-get-svc.png
    E03-ecr-repos.png
    E03-eks-dashboard.png
    E03-vpc-subnets.png
    E03-ec2-instances.png
    E03-iam-roles.png
    E08-aws-billing.png
```

E09-scale-to-zero.png
phase-3-ci/
E04-jenkinsfile.png
E04-shared-libraries-groovy.png
E04-security-gates-yaml.png
phase-4-gitops/
E05-gitops-repo-tree.png
E05-kustomization-yaml.png
E05-helm-chart.png
E05-argocd-project.png
E05-argocd-pods.png
E05-argocd-ui-dashboard.png
E07-kubectl-get-pods-retail-store.png
E07-kubectl-get-svc-retail-store.png
E07-32-pods-cluster.png
E07-argocd-synced-healthy.png
E07-jenkins-kaniko-builds.png
E07-ecr-images.png
E13-rollback-sequence.png
phase-4-security/
E10-kyverno-clusterpolicies.png
E10-kyverno-policyreport.png
E11-terrascan-scan.png
E11-terrascan-report-json.png
phase-4-observability/
E06-monitoring-pods.png
E06-monitoring-services.png
E06-cloudwatch-log-group.png
E06-grafana-dashboard.png
E12-prometheus-rules.png
E12-prometheus-alerts.png

Lưu ý:

- Đặt tên file theo format E<XX>-<mô tả ngắn>.png.
- Chụp toàn màn hình (Alt+PrintScreen), không crop ảnh để giữ context.
- Nếu evidence từ terminal, nên mở full-screen terminal, dùng font rõ ràng.
- Evidence E04 và E05 dùng VS Code dark theme để dễ đọc code.

- Evidence E08 nên có thêm export CSV từ Cost Explorer (nếu có).
- Evidence E10 và E11 (Kyverno + Terrascan) thuộc phase mới: `phase-4-security/`.
- Evidence E12 (Prometheus alerts) bổ sung cho `phase-4-observability/`.
- Evidence E13 (rollback) là minh chứng cho toàn bộ GitOps workflow.